ARM INSTRUCTION SET ENERGY MODELS

&

POWER SIMULATOR TOOLS (ARM7TDMI)

X.Amela, J.Figueras, S.Manich, J.Rius, R.Rodriguez, A.Rubio

UNIVERSITAT POLITÈCNICA DE CATALUNYA

BARCELONA, MARCH 2001

INDEX

Instruction Set Energy Models & Power Simulator Tools (ARM7TDMI)

Index 2

1 Objective Of This Document 2

2 Consumption In Microprocessors: State Of The Art 2

2.1 Energy Estimation Models 2
2.1.1 Classification of Models 2
2.1.2 Existing Models 2

2.2 Energy Estimation Tools 2

3 Proposals 2

3.1 Introduction 2

3.2 Simple Bus Cycle Energy Model 2
3.2.1 Introduction 2
3.2.2 Construction of the Model: The CYCLEPROCESS Tool 2
3.2.3 Construction of the Model: The MEASUREMENT Tool 2
3.2.4 Construction of the Model: The REGRESS Tool. 2
3.2.5 Summary 2

3.3 Enhanced Cycle Energy Model 2
3.3.1 Introduction 2
3.3.2 Approach: Considering Different Kind Of Cycles 2
3.3.3 Pipeline Reconstruction Tool 2
3.3.4 Summary 2

3.4 Microarchitectural Cycle Energy Model For ARM Processor 2
3.4.1 Introduction 2
3.4.2 Internal Architecture Of The ARM Processor 2
3.4.3 Description Of Basic Cycles 2
3.4.4 List Of Resources Activated By An Elementary Operation 2
3.4.4.1 Actions For An Address Calculation (LDR and STR) 2
3.4.4.2 Actions For A Data Transfer (LDR) 2
3.4.4.3 Actions For An Internal Cycle (MI LDR) 2
3.4.4.4 Actions For A Data Transfer (STR) 2
3.4.4.5 FETCH 2
3.4.4.6 Address Control (Fetch) 2
3.4.4.7 DECODE 2
3.4.4.8 EXECUTE 2
3.4.5 Energy Consumption Of The Principal Resources 2
3.4.5.1 Busses 2
3.4.5.2 Registers Write 2
3.4.5.3 Multiplier 2
3.4.5.4 ALU 2
3.4.5.5 Barrel Shifter 2
3.4.5.6 Decoder 2
3.4.5.7 Adjust Techniques With The Help Of Experimental Results 2
3.4.6 Summary 2

4 Results And Experimentation 2

4.1 Introduction 2

4.2 ARM Tools 2
4.2.1 ARMulator & Tracer 2
4.2.2 Trace Format 2

4.3 Measuring System 2
4.3.1 Components Of The Measuring System 2
4.3.1.1 AT91EB01 Evaluation Board 2
4.3.1.2 P6284 Differential Probe [TEK00a] 2
4.3.1.3 TDS744A Digital Storage Oscilloscope 2
4.3.1.4 TLA613 Logic Analyzer 2
4.3.1.5 ARMULATOR, MATLAB, LABVIEW And Dedicated Software 2
4.3.2 Measurement Method 2

4.4 Results 2
4.4.1 Introduction 2
4.4.2 Simple Bus Cycle Energy Estimator 2
4.4.2.1 Reliability Of The Coefficients 2
4.4.2.2 Power Chronograms 2
4.4.2.3 Summary 2
4.4.3 Enhanced Cycle Energy Estimator 2
4.4.3.1 Pipeline Reconstruction Tool 2
4.4.3.2 Summary 2

5 Conclusions 2

6 Bibliography 2

1 OBJECTIVE OF THIS DOCUMENT

This document embraces the results corresponding to deliverables 4.3 *Instruction Set Energy Models* and 4.4 *Power Simulator Tools*, this last partially because of the unexpected end of the project, being dedicated to the ARM7TDMI processor. Because of the partiality of the deliverable 4.4 the authors have decided to publish both deliverables in a single document in order to offer a comprehensive framework.

Section 2 *Consumption in Microprocessors: State of the Art* is an introduction (common to both deliverables) about the requirements and existing models to evaluate at high level (instruction level) the consumption of microprocessor-based systems and nowadays accessible tools. As the main contain of deliverable 4.3 section 3 *Proposals* introduces three instruction level energy models: *Simple Cycle Energy Model* that evaluates energy just from the number and type of processor cycles, the *Enhanced Cycle Energy Estimator* where the internal pipeline operation of the ARM processor is taken into account and the *Microarchitectural Cycle Energy Model* where the data and the internal process is evaluated. The three models offer three different levels of accuracy of the consumption.

Section 4, the main contain of deliverable 4.4 shows experimental results based on the three models proposed in previous section. For the two firsts the evaluating methods and tools as well as experimental results obtained in an energy measurement environment specifically designed for VIP are presented. For the third and more accurate model, an evaluating tool based on this technique is presented. Finally Section 5 summarizes the main results of the work, common to both deliverables.

2 CONSUMPTION IN MICROPROCESSORS: STATE OF THE ART

2.1 ENERGY ESTIMATION MODELS

2.1.1 CLASSIFICATION OF MODELS

During the last decade a large activity in the energy consumption modelling of digital circuits has been carried out in the research community. Initially, modelling was made at an electrical level because of the small complexity of the circuits. However recently, the interest to model larger systems like microprocessors has increased, pushed by the need to make more competitive systems. Microprocessors are large and complex systems that make difficult the modelling strategies used at the early years of the decade, based on electrical models. This is the reason why the research community has been proposing new kind of models and tools to cope with this problem **NEB96a**.

From a global point of view, any technique tries to find a trade-off between speed and precision that covers the requirements of a certain domain. For example, computer architects may require fast energy estimators to evaluate different design possibilities at expenses of low precision. However, quality engineers may require more precise energy models to check energy budgets for a given application. In general, the set of energy models may be located in a speed-precision hyperbole-like curve where larger speed means lower precision and upside down, see Figure 1.

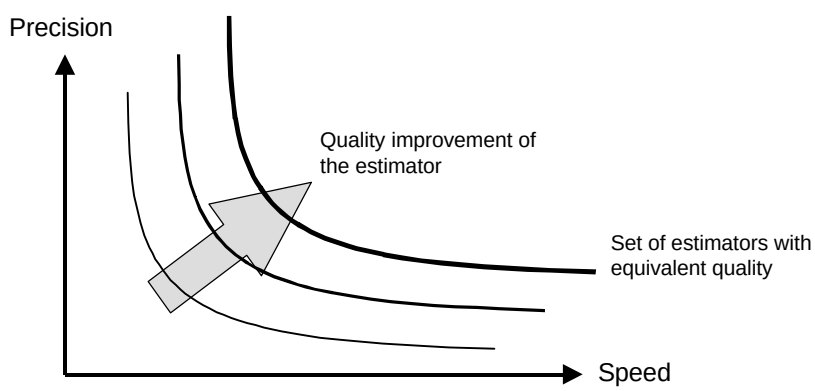


Figure 1: Speed-precision tradeoff hyperbole of energy estimators.

In other situations like for example in the early stages of a design, the lack of technological information may make impossible the construction of precise models. However, even in this limited case, the effort to make a fast energy estimator may become and attractive goal by itself.

Former figure qualitatively represents the evolution of this trade-off. If quality means quantity of scientific and algorithmic resources devoted to the design of an estimator, the set of equivalent quality estimators may be represented by a single hyperbole-like line in the speed-precision trade-off plot. Accordingly, estimators designed with more scientific and algorithmic knowledge, say better quality, belong to higher hyperbole-like line in the plot. Therefore, it is possible to improve precision without penalty in speed by improving quality.

In order to present a summary on different energy estimation techniques, a classification is made. Different criteria may be followed but the most commonly accepted is:

- The level of circuit abstraction used by the models.
- The use made by models of data processed by the circuit.

Figure 2 shows graphically the relation between these two criteria.

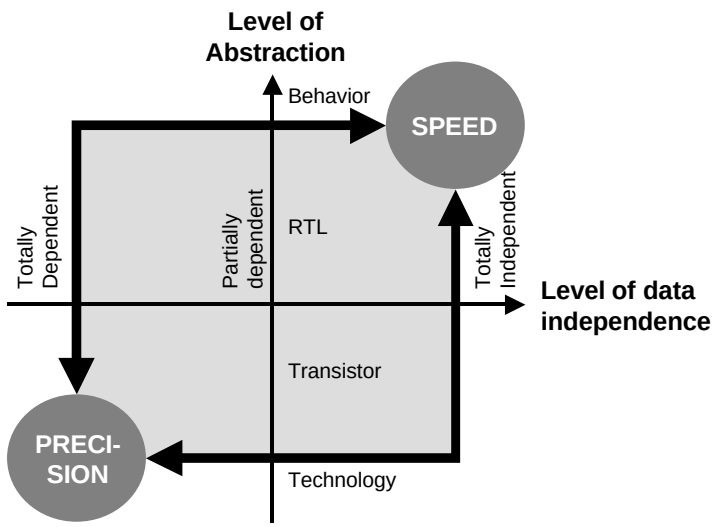


Figure 2: General classification criteria of energy estimation techniques.

The most precise models use all kind of information available from the circuit, i.e., technology (devices), structural (interconnection), and the data being processed by the circuit. These models require time-consuming simulations and are usually used at the final stages of the design. In earlier stages, faster models are required and this is the reason why two directions are followed to accelerate the estimations. First, models may be made exhibit weak data dependence. This is achieved by including statistical and probabilistic information in the model. The second alternative is a simplification of the expressions by lumping the technologic and structural information into few parameters. Usually a combination of both strategies is applied.

In the specific domain of microprocessors, this tradeoff between speed and precision is also found in the

algorithmic domain. Therefore, the concepts concerning models explained before can be fully translated to the energy consumption estimation of algorithms, which is the subject treated in this document.

Figure 3 shows the most common types of models used to estimate energy consumption in microprocessors.

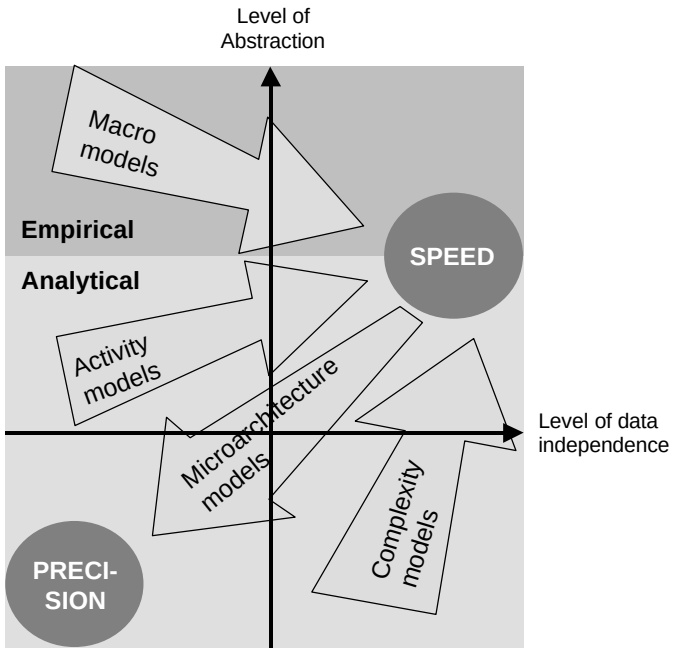


Figure 3: Types of models found in microprocessor's energy consumption estimators.

Two main groups are found **LAN96a**: the *analytical* and the *empirical*. Inside the group of analytical models three types can be found. The *complexity models* that extract physical information from the circuit to estimate average energy consumption. The *activity models* that take the entropy from data to similarly estimate average energy consumption. Finally, the *microarchitecture models* are top-down models that starting from a high level description of the circuit, an improvement of the precision is achieved by using designer knowledge and partial circuit information. The objective of these microarchitecture models is the prediction of the energy time domain.

Inside the empirical group it is found all the *macro-models* constructed from experimental data observation. The precision degree of these empirical models depends on the data information included in the macro-model. In spite of their general use is the prediction of the average energy consumption, they can be improved to predict the time domain of the energy.

From these four categories of models the most currently found in energy estimation of algorithms are the macro-models and the microarchitecture models. Actually, a combination of both is a typical situation. The reason is because microprocessor designers and energy model engineers use to be different work groups, and thus, in many cases it will become expensive the access to full microprocessor information.

As it is illustrated in Figure 4, the normal pathway to construct the model begins with macro-modelling. The

macro-model obtained can be very accurate for the experimented microprocessor but at the same time very sensitive to any circuit change. In order to make the model less sensitive, microarchitecture strategies may be applied.

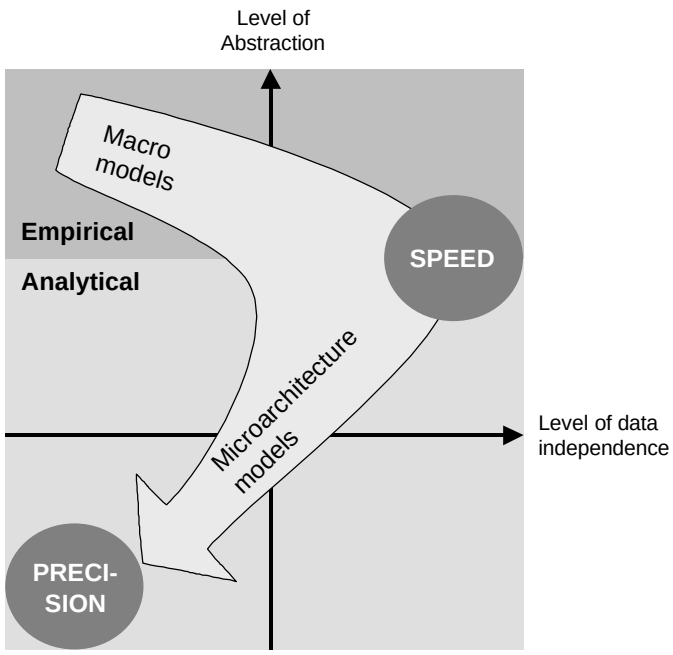


Figure 4: Common modelling methodology in energy estimation of algorithms.

Certain degree of technological and structural information is introduced in the model so much the energy estimations become less sensitive to changes in the circuit.

Following a brief description of existing models is presented.

2.1.2 EXISTING MODELS

The most detailed models used in digital systems to estimate the energy consumption are the electrical models (SPICE). In this level of detail, all the components responsible of energy consumption are considered **CHA92a GU95a RAB96a**, say

Switching capacitive component, E_C

Short-circuit component, E_{SC}

Static component, E_{ST}

Leakage component, E_L

Parameters E_C and E_{SC} are named dynamic components while E_{ST} and E_L are named static components. In general, approximated models assume E_C be the most important cause of energy consumption. The E_{SC} component is usually neglected or approximated to a factor proportional to E_{SC}. The static component E_{ST} is not considered in CMOS circuits. Finally, the term E_L is mostly neglected except for very deep submicron technologies.

The earliest proposed techniques of estimating power dissipation were strongly pattern-dependent circuit simulation based **KAN86a YAC89a NAJ94b**. Electrical current models are used during simulation and voltage and current waveforms are monitored. Besides being strongly data dependent, these techniques are too slow to be used on large circuits, for which high power dissipation is a problem.

In order to improve the computational efficiency, other simulation based techniques has been proposed using various kinds of timing, switch-level, gate level, and logic simulation **GU95a TUR95b DAG97a DEN95a TJA89a MAN95a MAN98a JAG90a KRO91a**. These techniques generally assume that the power supply and ground voltages are fixed, and the supply current waveform or the charge switched by the logic elements is estimated.

Models at RTL level of abstraction are commonly used in microprocessor energy estimation when high precision is not a strong requirement. At this level, the switching capacitor component of the energy is assumed the main factor. However, in some cases, the static component is considered just as a constant factor added to the total energy.

Models working at an RTL level of abstraction are for example the proposal of K. Müller *et al.* **MUL91a**. This model is analytical of the type complexity model. It computes average power consumption that is proportional to the number of gates of the circuit. The number of gates is approximated by identifying the function of the circuit or by means of a library. Gates are assumed to have two inputs. In the expression proposed by Müller, an activity factor is present that indicates the average number of gate commutations compared to the frequency of the clock. This activity factor synthesizes the influence of the data in the circuit. One disadvantage of this model is that all power estimates are based on the energy consumption of a single reference gate. This does not take into account different circuit styles, clocking strategies, or layout techniques. The approximation is particularly inaccurate for specialized blocks such as memories.

Previous limitation is improved in the model proposed by Liu *et al.* **LIU94a**. They design specific models for different blocks of the microprocessor. This has de advantage that the model includes de particularities of the circuit, especially if it has a very well known and repetitive structure. The logic component of power is estimated in a manner conceptually similar to the Müller model. The basic switching energy uses a three input AND gate, calculate from fundamental technology parameters. The total chip logic power is estimated (as before) by multiplying the estimated gate equivalent count by the basic gate energy and the activity factor. The activity factor is provided by the user and assumed fixed across the entire chip.

Previous complexity models have the advantage that they require very little information: few technology parameters, memory sizes and equivalent gate's count. One disadvantage is found in how they include information from the data. All this information is clustered into the activity factor, which is a very rough approximation of the influence the internal activity of the nodes has in the energy consumption.

Najm NAJ95a uses a different strategy. His model is analytical of the type activity based model. Here, the concept of entropy extracted from the information theory is treated. This can be used as a measure to estimate the average switching activity in a circuit **NAJ95a MAC95a**. The underground idea is relate the energy consumption to the computation work made by the circuit. Najm, observes that power is proportional to the product of physical capacitance and activity. Physical capacitance can be related to circuit area and activity to entropy. Thus the product of area and entropy can be assumed proportional to average power. Estimations of the circuit area have been proposed in the past **PIP77a CHE90a**. It is related to the number of Boolean inputs and the total entropy of outputs. Since an approximation of quadratic decrease of entropy according logic depth is used, the entropy of all the gates can be estimated as a function of input and output entropy. In summary, the function of the circuit and the input and output entropy can be used to estimate the average power consumption. In the proposal of Najm, the input and output entropy are obtained from an RTL simulator.

The author notes that limitations of this model exist because time domain is not modeled. Therefore the internal glitching is not included in the estimations. Another limitation is the assumption of the uniform distribution of the parasitic capacitance of the circuit.

As has been presented so far, the accuracy of these techniques is limited, however they have been proved useful in relative comparison of architectures. Anyway, the strong assumptions made by these models make their energy or power predictions far from real hardware.

Powell *et al.* **POW90a** proposes an empirical model constructed by macro-modelling named Power Factor Approximation (PFA). Instead of using architectural information described at RTL level and trying to relate to technological information, real energy measurements are made. An approximation is taken to relate power to hardware complexity and activation frequency. This parameter of complexity is for example in a multiplier the number of input bits squared. The activation is the frequency the multiply is used for a given algorithm. A proportional factor adjusts these two magnitudes to the empirical measures. This model is best suited for characterization of libraries, especially for blocks like memories and I/O drivers. The limitation of the model is the fixed activity assumption. Blocks like multipliers may give large errors since they are very sensitive to input data.

Sato *et al.* proposes an improvement to the previous model since his model includes some dependence on input data. His model is fundamentally a cycle-based simulator targeted at a RISC processor. As object code is executed, the tool including the model monitors which blocks in the architecture are activated. This is an indication to accumulate a fixed contribution to the total energy consumption for each module. This fixed quantity is empirically adjusted previous simulation. The datapath power model accounts to some extent for input vector activity by using a power model that has a constant portion and a portion that is proportional to the number of bit transitions in the input vector.

Another experimental model accounting for input data activity is the proposal of Landman *et al.* **LAN94a LAN95a LAN95b**. Here, previous the construction of the model, data activity is recorded using a simulator for given profiles of data. After simulation, statistics are extracted from data activity that is included in the models of different blocks. Basically two blocks are described: datapath and controlpath. In the datapath block, high activity bits and low activity bits are separated in two different models since the observation displays a different behavior. In one case data is highly correlated while in the other the correlation is very low. Fitting parameters are defined individual for each group of bits giving different behavior. In the controlpath, transition probability and signal probability **NAJ92a NAJ94b XAK94a** is combined with complexity parameters and fitting models to estimate the energy in various controller implementations.

A recently contribution on this type of model is the proposal of Brooks *et al.* **BRO00a** that specifies models for different type of typical microprocessor blocs. Here, a behavior for power modeling is presented which systematize the extraction of technological parameters. Hence, models can be parameterized to allow technological changes.

In spite this model has a very good agreement with switching-level models, applications where the type of data patterns processed by the microprocessor change may cause the estimation error increase significantly since models are very much specific for data execute previous de model fitting.

Models at higher abstraction levels have also been presented like the Mehra and Chandrakasan proposals **MEH94a CHA95c**. Here the strategy is to estimate the total energy consumption as a function of the access frequency of different hardware resources. This access frequency is calculated analyzing the behavioral description of the function to be implemented. The description could be in C, Verilog or VHDL. Since this analysis is performed with one pass through the program it is very fast. This access frequency is used to multiply the energy contribution of each resource. Individual energy contributions are hide into an average capacitor that is unique for each resource and it is obtained by experimental fitting using different configurations of the resource and different type of data patterns. Clearly, the goal of this type of model is to have an idea of what are the general trends of power for different system configurations. Absolute accuracy is not aimed.

In the previous case the data contribution is included in a static manner. Another proposal of Kumar *et al.* and San Martin *et al.* **NAN95a MAR96a** improves de data dependencies the access frequencies are computed in real time as the VHDL simulator executes the function of the system. A disadvantage of this strategy is the increase of the estimation time required for energy consumption computation.

A different conceptual behavior is the proposed by Tiwari *et al.* **TIW94b**. It is based in an empirical macromodelling approach where each available instruction of the microprocessor is placed in a loop and executed on the target processor. During this process current measurements are taken, and the average current drawn by each instruction is stored in a table of base costs. The model also handles what are referred to as inter-instruction effects. This model has been used to characterize the Intel DX2 processor, the Fujitsu SPARClite 934 and a Fujitsu embedded DSP processor. The authors note that while accurate for most instructions, the estimates can error significantly in certain arithmetic instructions like multiplication. This is caused by the large dependence on data of this type of block. A systematic methodology to fit instruction energy of microcontroller is presented by **CHA99a**.

In the proposal of Lee *et al.* **LEE97a TIW98a** this constrain is improved. A microarchitecture model of the multiplier is defined and an RTL model is defined. This model is described at half and full adder level and average capacitance of each type of block is defined. A significant improvement of the error is achieved by this procedure.

More recently, Conte *et al.* **CON00a** has improved the instruction level model by increasing its precision. They have modeled the inter-instruction influences by splitting the processor execution steps into the pipeline cycles. Using this, they are able to better associate algorithm execution with technology since they can relate in a temporal domain the access of different hardware units.

2.2 ENERGY ESTIMATION TOOLS

At the present moment a variety of energy estimation tools exist. They are based in models which type has been presented in the previous section. Then, in order to avoid repetitions a table is presented that lists the different tools and the type of models included into that.

Tool	Description
SPICE	Electrical Simulator **NAG73a**
PowerMill	Switch Level Simulator **HUA95a**
SIMCURRENT	Switch Level Simulator **JAG90a**
PowerPlay	Logic Level Simulator **KRO91a**
SAIL	Logic Level Simulator **MAN99a**
McPower	Monte Carlo Tool based on Probabilistic Models **BUR92a**
CES	Complexity Based Models **MUL91a**
ESP	Cycled Based Estimator Tool **SAT95a**
SPA	Architectural Sensitive Power Analysis. Simulation Based Activity Profiler **LAN94a**
WattWacher	Simulation Based Activity Profiler **WAT96a**
Hyper-LP	High Level Synthesis System **MEH94a CHA95c**
Wattch	Framework for Architectural-Level Power Analysis and Optimization **BRO00a**

Table 1: Energy estimation and simulation tools.

3 PROPOSALS

3.1 INTRODUCTION

In this chapter we first present a *Simple Bus Cycle Energy Model*, a very simple consumption model of the microprocessor built by identifying the bus cycles executed by the microprocessor and computing the consumption for each one of these bus cycles. Later, the *Enhanced Cycle Energy Model* will be introduced. This model will use a more complex model, including a cycle-by-cycle pipeline activity reconstruction. Finally a third energy model for the instructions of the ARM processor will be presented. In this case both the type of instruction as well as the operand's data is taking into account under the assumption of a microarchitecture model of the processor; with this strategy a more accurate cycle energy model is derived allowing the consideration of voltage as well as technology trends of the implementing VLSI technology.

3.2 SIMPLE BUS CYCLE ENERGY MODEL

3.2.1 INTRODUCTION

The consumption of a microprocessor may be modeled by identifying their active parts along the time, and assigning to each one of these active parts a given weight. This weight packs in a single number the contribution of each active part to the total consumption, which is computed by adding all the contributions. On the other hand, the activity in the address and data buses has a strong correlation with the microprocessor consumption. Therefore, a very simple consumption model of the microprocessor may be built by identifying the bus cycles executed by the microprocessor and computing the consumption for each one of these bus cycles.

The ARM7TDMI microprocessor has a pipeline of three stages (FETCH, DECODE and EXECUTE) and is able to execute simple instructions in a single clock cycle when these instructions are fetched from a memory connected to the microprocessor address and data buses. Thus, in the clock cycle (i), the ARM7TDMI typically fetch the instruction (i), decodes the instruction (i-1) and execute the instruction (i-2), holding the pipeline full. If this is the case, from the point of view of the bus activity, the microprocessor is performing a burst of fetch cycles. The load/store instructions break the continuity of the pipeline because they use the address and data buses to perform the data transfer, thus introducing bubbles in the pipeline and producing the so called INTERNAL cycles. Other instructions may also introduce INTERNAL cycles, like MULTIPLY or LDM/STM (load/store multiple).

In this way, we can recognize in the ARM7TDMI microprocessor the following types of bus cycles:

- FETCH: during this cycle an instruction is fetched from the memory
- LOAD: data is loaded to a register from memory.

- STORE: data is stored to memory from a register.
- INTERNAL: the ARM7 is executing an internal operation.

As it is explained above, during each cycle the ARM7 is performing more operations in parallel than the contained in this listing. However, these four types of cycles are enough to make the simple energy model we are describing here. By assigning a weight (Joules/cycle) to each type of cycle we are able to estimate the total energy and also the evolution (in a cycle-by-cycle basis) of the energy consumed by the ARM7. Notice that this model can be used for either the THUMB or ARM instruction set.

Then, the first step to construct the model is to identify the bus cycles executed by the ARM7 during the execution of a given program. This is the goal of the **CYCLEPROCESS** tool developed in this project.

3.2.2 CONSTRUCTION OF THE MODEL: THE CYCLEPROCESS TOOL

The "**CYCLEPROCESS**" is the program that permits us counting the real number of microprocessor cycles present in an ARM/THUMB routine. Behind this program there is knowledge and an investigation about the real behavior of the ARM7TDMI microprocessor and the number of clock cycles taken during the execution of a given instruction type. This aspect has been studied through a logic analyzer that has solved this critic question. Thanks to this analyzer we have found the following relations about each instruction and its clock duration in function of the number of bits manipulated and the program memory location.

The program memory location has been taken into account because the number of clock cycles needed to execute completely each instruction (FETCH, DECODE and EXECUTE it) depends on the location of this memory. Two possibilities have been studied:

- **Internal** memory: the program memory is connected directly to the ARM7 data and address busses (32 bits).
- **External** memory: the memory is connected to the ARM7 through an external data bus interface 16 bit wide.

The number of clock cycles for each access to Internal/external memory is the following:

- Number of clock cycles on External Memory:

Bus cycle	Data Manipulation		
	8 bits	16 bits	32 bits
Fetch*	-	2	4
Data Read	2	2	4
Data Store	2	2	4
Internal Cycle	1	1	1

- Number of clock cycles on Internal Memory:

Bus cycle	Data Manipulation		
	8 bits	16 bits	32 bits
Fetch*	-	1	1
Data Read	1	1	1
Data Store	1	1	1
Internal Cycle	1	1	1

* In the fetch, the 16 or 32 bits data manipulation is the same that a fetch of THUMB or ARM instruction, respectively.

So, we can say, for example, that a LDR (Load Register) instruction of 32 bits, takes 9 clock cycles on external memory and ARM instruction type (4 cycles for the Fetch, 4 for the data read and 1 for an Internal cycle; the execution is transparent due to the fetch pipelining of the another instruction).

Using the output of the debugging tracer from the ARMULATOR tool, we can know the number of clock cycles executed by the microprocessor. This is exactly what the "CYCLEPROCESS" does; it has an internal database with all the bus cycle types, their clock cycle duration and its description present in the debugging tracer. During the execution of the "CYCLEPROCESS", it reads step by step the file containing the debugging tracer of a routine. First, in each step, a type of bus cycle is read, second, this is compared with the types in the database and finally it is assigned the real duration of the bus cycle. At the end of the program the total number of clock cycles is also counted.

After this explanation, we can see two text fragments: the first belongs to a debug tracer sample, the input of the "CYCLEPROCESS" program. Here, we can see that first word of each row describes the type of bus cycle; this is what the "CYCLEPROCESS" scan to count the clock cycles. To know if the scanned routine runs on internal or external memory, the "CYCLEPROCESS" scan the next number that refers to the memory position. For example, MNR4O refers to a 4 bytes fetch, and 02000000 encode an external memory position, so the "CYCLEPROCESS" will assign 4 clock cycles to this fetch.

```
Date: Tue Feb 20 20:46:09 2001
Source: Armul
Options: Trace Instructions (Disassemble) Trace Memory Cycles (Idles) Trace
Events

MNR4O__ 02000000 E3A00885
MSR4O__ 02000004 E59F1098
MSR4O__ 02000008 E5810000
IT      02000000 e3a00885 MOV r0, #0x850000
MSR4O__ 0200000C E3A00885
IT      02000004 e59f1098 LDR r1, 0x20000a4
MNR4___ 020000A4 FFFF0000
MI
MSR4O__ 02000010 E59F1090
IT      02000008 e5810000 STR r0, [r1, #0]
MNW4___ FFFF0000 00850000
```

```
MNR4O__ 02000014 E5810000
IT      0200000C e3a00885 MOV r0, #0x850000
MSR4O__ 02000018 E3A00010
IT      02000010 e59f1090 LDR r1, 0x20000a8
MNR4___ 020000A8 FFFF0010
MI
MSR4O__ 0200001C E59F1080
IT      02000014 e5810000 STR r0, [r1, #0]
MNW4___ FFFF0010 00850000
MNR4O__ 02000020 E5810000
IT      02000018 e3a00010 MOV r0, #0x10
MSR4O__ 02000024 E3A00010
IT      0200001C e59f1080 LDR r1, 0x20000a4
MNR4___ 020000A4 FFFF000     ....
```

The second text refers to an output of the "CYCLEPROCESS". These details all types of bus cycles, how many times they appear in the scanned routine, the total of bus cycles and the total number of clock cycles. Here we can see that the scanned routine has run on internal memory and all the bus cycles takes only one clock cycle.

```
          *** Results ***
Fetch External memory 16 bit (x2): ..........0
Fetch External memory 32 bit (x4): ..........0
Fetch Internal memory 16 bit (x1): ..........0
Fetch Internal memory 32 bit (x1): ..........670
Read External memory 8 bit (x2): ...........0
Read External memory 16 bit (x2): ...........0
Read External memory 32 bit (x4): ...........0
Read Internal memory 8 bit (x1): ...........0
Read Internal memory 16 bit (x1): ...........0
Read Internal memory 32 bit (x1): ...........197
Write External memory 8 bit (x2): ...........0
Write External memory 16 bit (x2): ..........0
Write External memory 32 bit (x4): ..........0
Write Internal memory 8 bit (x1): ...........0
Write Internal memory 16 bit (x1): ..........0
Write Internal memory 32 bit (x1): ..........328
Internal Cycles ............................197

  Total Cycles:       1392
  Real Clock cycles:  1392
```

3.2.3 CONSTRUCTION OF THE MODEL: THE MEASUREMENT TOOL

After we obtain the number and type of bus cycles, the second steps is assigning a weight (Joules/cycle) to each bus cycle type. Measuring the ARM7 consumption when it runs multiple small training programs and correlating the measured consumption with the number and type of bus cycles executed has solved this problem.

We have used a particular implementation of the ARM7TDMI architecture: the AT91R40807 microcontroller and an Evaluation Board to perform these measurements. The details of such measurement are described in section 4.2.1.

3.2.4 CONSTRUCTION OF THE MODEL: THE REGRESS TOOL.

By running *m* small training programs and applying to each program the CYCLEPROCESS and MEASUREMENT tools, we obtain the following data:

- A matrix with *m* rows and 17 columns. Each column is devoted to a type of bus cycle.
- A vector of *m* elements each one of them is the energy measured of a given program.

This information is the input of the REGRESS tool, which performs a multiple linear regression process and extracts from the data a set of coefficients, which are the energy consumed in the execution of each type of bus cycle, thus ending the construction of the model.

3.2.5 SUMMARY

This is the framework of our first consumption model for the ARMTDMI microprocessor. Obviously, by using this model we lose information like, for instance, the particular instruction that has been executed, or the data value that has been stored. Also, the model depends on the particular implementation the microprocessor has been made. However, this simple model has the following advantages:

- After the coefficients have been computed, the consumption of a long program is estimated with a very good precision.
- It is easy to implement.
- It predicts not only the total energy consumed by a given program, but also the evolution of this consumption along the time in a cycle-by-cycle basis.

Some results showing how this simple model predicts the consumption of the ARM7TDMI will be shown in section 4.2.2.

3.3 ENHANCED CYCLE ENERGY MODEL

3.3.1 INTRODUCTION

Similarly to the proposal in section 3.2, a system to obtain the cycle-by-cycle program evolution consumption will be also proposed here, but, a more complex model, including a cycle-by-cycle pipeline activity reconstruction, will be used now. To do so, as in the previous section, a cycle matrix starting from the trace provided by the ARMulator's Tracer will be generated. This cycle matrix will allow connecting the processor's consumption when executes a program with the activity generated by the program into the core when every instruction is executed. Again, this information will be the input for a regression tool, which will perform a multiple linear regression process, extracting from the data a set of coefficients, which are the energy consumed in the execution for every cycle by all the different active blocks.

In the case we are dealing now, the information generated by the tracer will be, as in the previous section, the starting point. This information will allow to know the temporary evolution of the program and, therefore, the temporary sequence of all memory accesses (and, thus, also all fetches) and instruction executions; or, which is the same, from this trace we will be able of knowing when and what fetch is being generated (knowing, moreover, attributes like access address, instruction which is being fetched, etc.); when and what instruction is being decoded; and when and what instruction is being executed (with its respective read/write memory accesses characterized, and/or its respective internal execution cycles, in case they exist). Processing in an appropriate way this information will allow reconstructing the cycle-by-cycle pipeline temporary evolution.

Once the pipeline evolution will have been reconstructed, we will know cycle-by-cycle what blocks are working in the processor and, if we already knew every block's consumption, for a given program execution we would be able of estimating the cycle-by-cycle processor consumption, or, in other words, the temporary consumption evolution.

3.3.2 APPROACH: CONSIDERING DIFFERENT KIND OF CYCLES

An approach to the idea is considering several different cycles and distinguishing each other according with coefficients which would reflect their attributes. That is, those attributes would distinguish if during such a cycle there were a fetch, and/or there is a decoding, and/or there is a memory access, and/or there is an instruction execution –making difference among different kind of instructions-, etc. Afterwards, connecting those types of cycle with the program consumption would allow estimating the consumption for each kind of cycle.

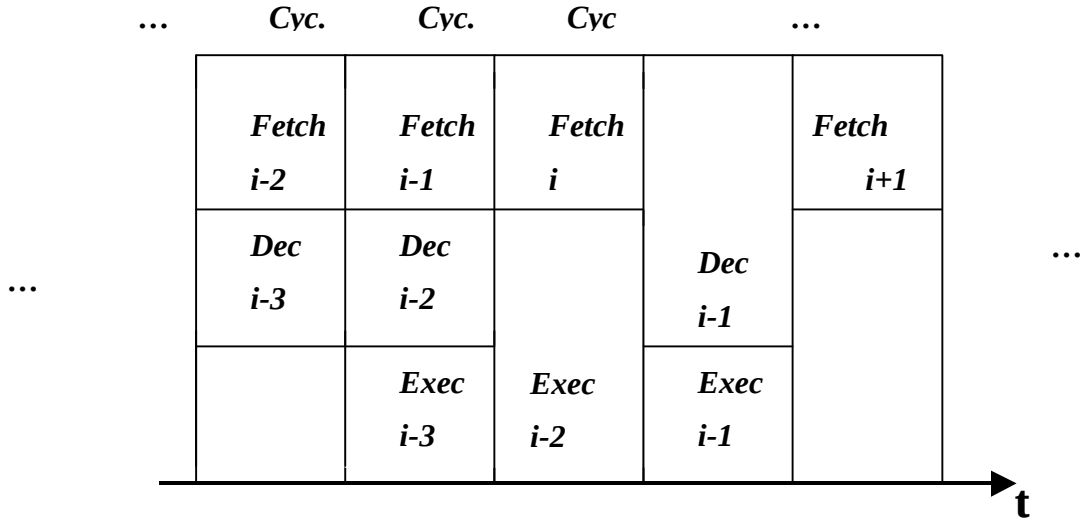
In this approach, where we will distinguish the different types of cycle, the parameters are:

- F: The processor carries out a fetch during the present cycle.
- DEC: The processor carries out a decoding during the present cycle.
- INT_MEM_ACC: The processor carries out an internal memory access during the present cycle.

- EXT_MEM_ACC: The processor carries out an external memory access during the present cycle.
- MIM: The processor carries out an internal multiplication cycle during the present cycle.
- MILS: The processor carries out an internal load cycle during the present cycle
- MIO: The processor carries out any other internal cycle
- DAT_PRC_INST: The processor executes a 'data processing' instruction during the present cycle.
- MULT_INST: The processor executes a 'multiply' instruction during the present cycle.
- MOV_INST: The processor executes a 'move' instruction during the present cycle.
- B_EXCH_INST: The processor executes a 'branch & exchange' instruction during the present cycle.
- LD_S_INST: The processor executes a 'load single data' instruction during the present cycle.
- ST_S_INST: The processor executes a 'store single data' instruction during the present cycle.
- SWAP_INST: The processor executes a 'swap' instruction during the present cycle.
- LD_M_INST: The processor executes a 'load multiple data' instruction during the present cycle.
- ST_M_INST: The processor executes a 'store multiple data' instruction during the present cycle.
- BRC_INST: The processor executes a 'branch' instruction during the present cycle.
- INT_INST: The processor executes a 'interruption' instruction during the present cycle.

Now, using those coefficients, we will be able of characterizing the different kind of cycles which are executed when the program is running and, knowing the processor blocks which are being make work by each component of each cycle, we will be able of estimating the total consumption during the cycle.

If, for instance, in an instant of time there was a cycle sequence like the one below:



Where, the consumption when a fetch takes place would be given by the F coefficient; the consumption during a decoding would be given by a coefficient D; and the consumption for the different instruction executions would be provided by some of all the other coefficients in the different instruction groups previously

discussed. Then, we can see in the Figure that there would be a cycle (Cycle i-1) in the program execution, which would be formed by a fetch (Fetch i-1) , a decoding (Dec i-2) and an execution (Exec i-3) ; where the coefficients corresponding to those actions would characterize the cycle. This cycle would be different from another one (Cycle i) formed, for instance, by a fetch (Fetch i) and a different execution (Exec i-2) , which would also be characterized by its respective coefficients. Therefore, adding the coefficients corresponding to the fetch, the decoding and the execution for our first cycle, we would obtain this cycle consumption; and it would be different from the cycle consumption obtained adding the coefficients corresponding to the fetch and the execution in our second cycle.

Considering a group of several programs, where several means a number much higher than the number of coefficients to characterize the cycles, and where the different kind of cycles which we can find are known; and knowing also every program total consumption, it is possible to do a multiple linear regression in order to get an approximate value for every kind of cycle weight. To carry out this operation we need a matrix with the number of times that each different kind of cycle has been executed for each program, and a vector with the total consumption of each program.

$$\begin{pmatrix} \text{Cycle 1 Times in Prog. 1} & \text{Cycle 2 Times in Prog. 1} & \text{Cycle 3 Times in Prog. 1} & \cdots \\ \text{Cycle 1 Times in Prog. 2} & \text{Cycle 2 Times in Prog. 2} & \text{Cycle 3 Times in Prog. 2} & \cdots \\ \text{Cycle 1 Times in Prog. 3} & \text{Cycle 2 Times in Prog. 3} & \text{Cycle 3 Times in Prog. 3} & \cdots \\ \vdots & \vdots & \vdots & \end{pmatrix} \quad \begin{pmatrix} \text{Consumption Program 1} \\ \text{Consumption Program 2} \\ \text{Consumption Program 3} \\ \vdots \end{pmatrix}$$

Moreover, as we will already know the program total consumption, considering the same group of programs and knowing how many times every factor, from the ones determining the different cycle types, has happened during the program execution, we will also be able of doing a multiple linear regression; so that we will obtain a value for the weigh that will characterize every coefficient in the total consumption routine. In order to do this estimation we will need a matrix with the information of how many times each parameter has taken place for each program, and a vector with each program consumption.

Finally, it is possible to link the results from both regressions (the approximate value for every coefficient which will characterize the cycles, and the approximate value for every cycle type), in such a way that the different cycle consumption has to be the same as the total of adding each factor characterizing the cycle.

However, as we have already said before, the final aim for the estimator is to estimate the consumption evolution starting from the different parts in the core which are excited during each cycle and, thus, those coefficients will not be calculated with the multiple linear regressions, but they will be calculated knowing the activity generated in the active processor parts during each cycle. On the other hand, those regressions will help us to tune the final theoretic value, and they could be considered like a train for the system.

3.3.3 PIPELINE RECONSTRUCTION TOOL

The Pipeline Reconstruction Tool is the program that will allow obtaining, once the routine under test has been compiled and its trace extracted, all the memory accesses and instruction executions from this trace. Behind this program, as in previous tools, there is knowledge about the real behavior of the ARM7TDMI microprocessor and the number of clock cycles taken during the execution for a given instruction; so that with this information extracted from the trace and with our knowledge about the microprocessor, we will be able of obtaining the cycle-by-cycle pipeline evolution.

By running the programs and applying the Tool, we will obtain not only the pipeline evolution but also the information which will allow to characterize the cycles, so that we will obtain the number of fetches, decoding, different executions what have happened, and the different type of cycles (with their characterizing coefficients) that took place during the program execution. This information will be later used in the matrix to perform the multiple linear regressions to extract from the data a set of weights for the coefficients, which are a first estimation for the energy consumed in the execution of each type cycle, or will be used together with the microarchitectural model explained in section 3.

Since the aim of the present point is to present the model and not to present the way to obtain the cycle matrix and its qualities, the method to calculate the regression starting from the cycle matrix will be detailed later on, in section 4. On the other hand, the method to obtain the consumption programs vector will be also explained in the same section, devoted to the Results and Experimentation.

3.3.4 SUMMARY

Summarizing the method presented in this section, a system to obtain the cycle-by-cycle consumption program evolution has been proposed. This system will use a more complex consumption model than in previous sections, including a cycle-by-cycle pipeline activity reconstruction. Using this model, some information like the data processed or the processor architecture are missing, so it will not be as accurate as the next model presented; but despite so, this technique takes into consideration more information than just the bus cycle, since it is able of differentiating the active blocks during every cycle execution. Once the coefficients are computed, via statistics or with a more accurate model, the consumption evolution for a program is estimated with a pretty nice precision. Some examples and results will be shown later, in section 4.

3.4 MICROARCHITECTURAL CYCLE ENERGY MODEL FOR ARM PROCESSOR

3.4.1 INTRODUCTION

In this section a third energy model for the instructions of the ARM processor is presented. Since the final aim for the estimator is to estimate the consumption evolution starting from the different parts in the core which are excited during every cycle, the coefficients presented until this section are not going to be calculated with multiple linear regressions, but they are going to be calculated bearing in mind the activity generated in the active processor parts during the cycles. On the other hand, and anyway, those regressions will help us to tune the final theoretic value, and they could be considered like training for the system.

In this case both the type of instruction as well as the operand's data are taking into account under the assumption of a microarchitecture model of the processor.

With this strategy a more accurate cycle energy model should be derived allowing the consideration of voltage as well as technology trends of the implementing VLSI technology.

In section 3.4.2 a description of the internal architecture of the ARM processor is presented. The basic cycles of the instructions are described in 3.4.3. From this information for each instruction of the ARM set a list of resources that are activated cycle by cycle in each instruction is generated. Several examples for basic instructions are presented in section 3.4.4. For each resource activated a consumed energy model is derived (section 3.4.5). This energy model is data sensitive, this means that the data transferred through the busses and the data processed by the respective blocks are taking into account. This implies that the Energy Evaluation Tool (see section 4.3) stores the status of the registers and busses. For each instruction the total energy consumed is evaluated adding the energy consumption of all the resources activated by the instruction. Finally in section 3.4.6 the experimental actions oriented to adjust the model to a given technology and implementation are commented.

3.4.2 INTERNAL ARCHITECTURE OF THE ARM PROCESSOR

Basically the ARM processor is a RISC type processor which basic structure is shown in Figure 5. The processor communicates externally through an Address and Data Bus. Internally there is a process unit, common to data and address calculation, with advanced ALU and multiplier an instruction decoder with its control system based on a three level pipeline system. At each cycle the processor operates (at maximum) a fetch, a decoding and an execution action. The processor incorporates a 64-bit register bank.

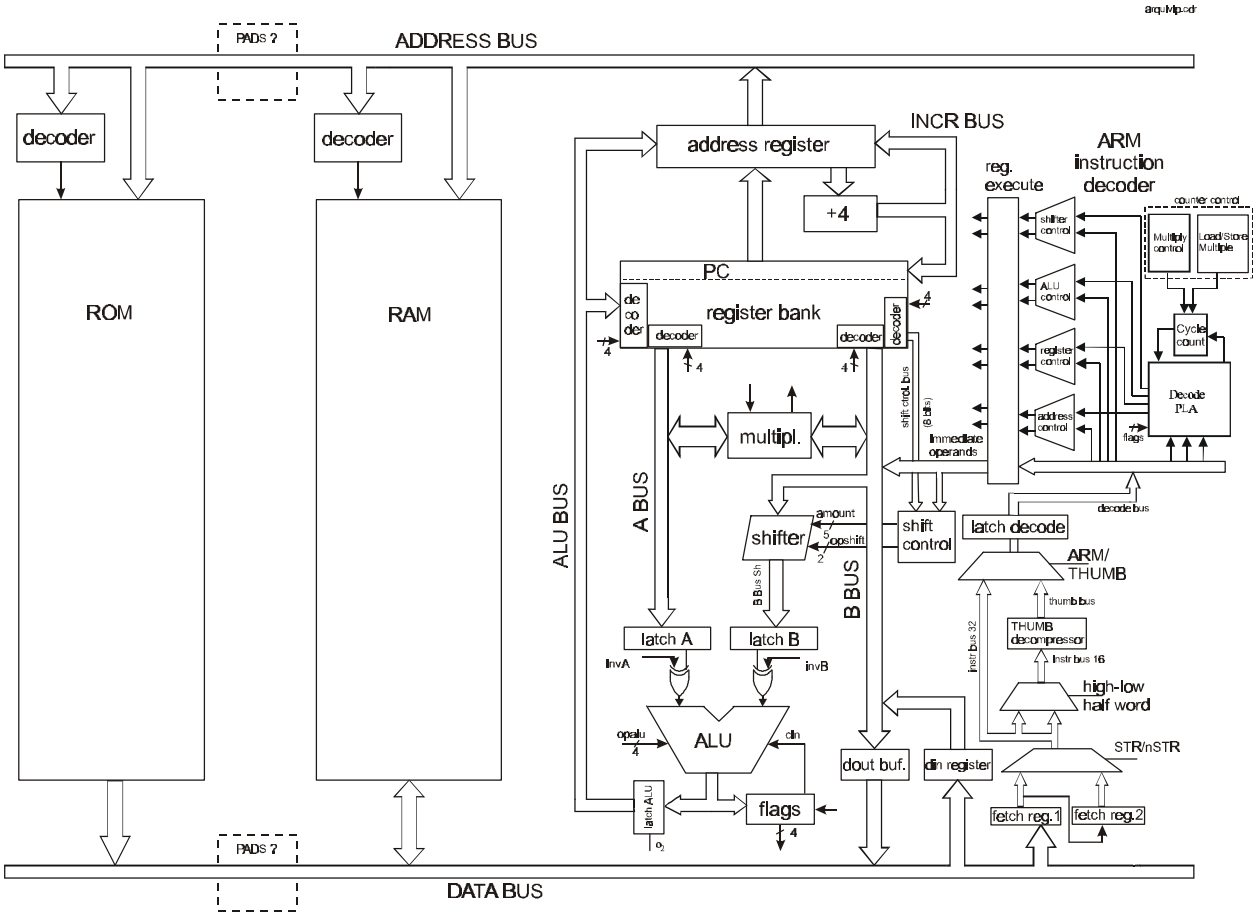


Figure 5: ARM processor basic structure.

3.4.3 DESCRIPTION OF BASIC CYCLES

It is possible to show on this architecture the flux of data and addresses for each instruction for the three potential concurrent actions. Figure 6, Figure 7 and Figure 8 show the transfer data basic cycles for the ADR and STR instructions.

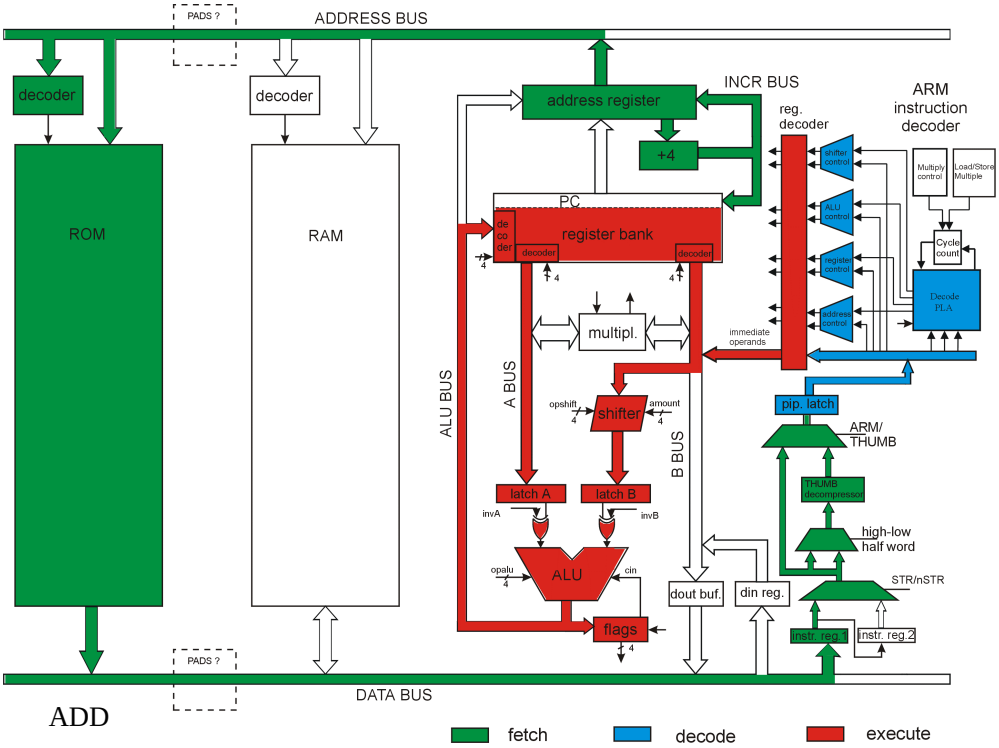


Figure 6: Instruction ADD execution.

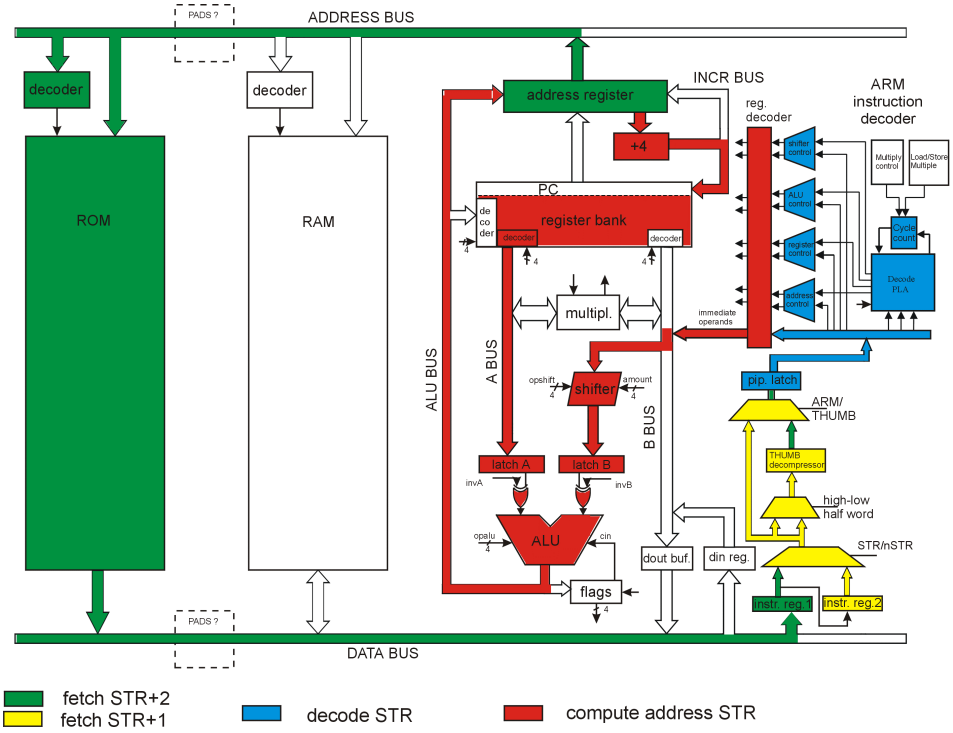


Figure 7: First cycle of instruction STR execution.

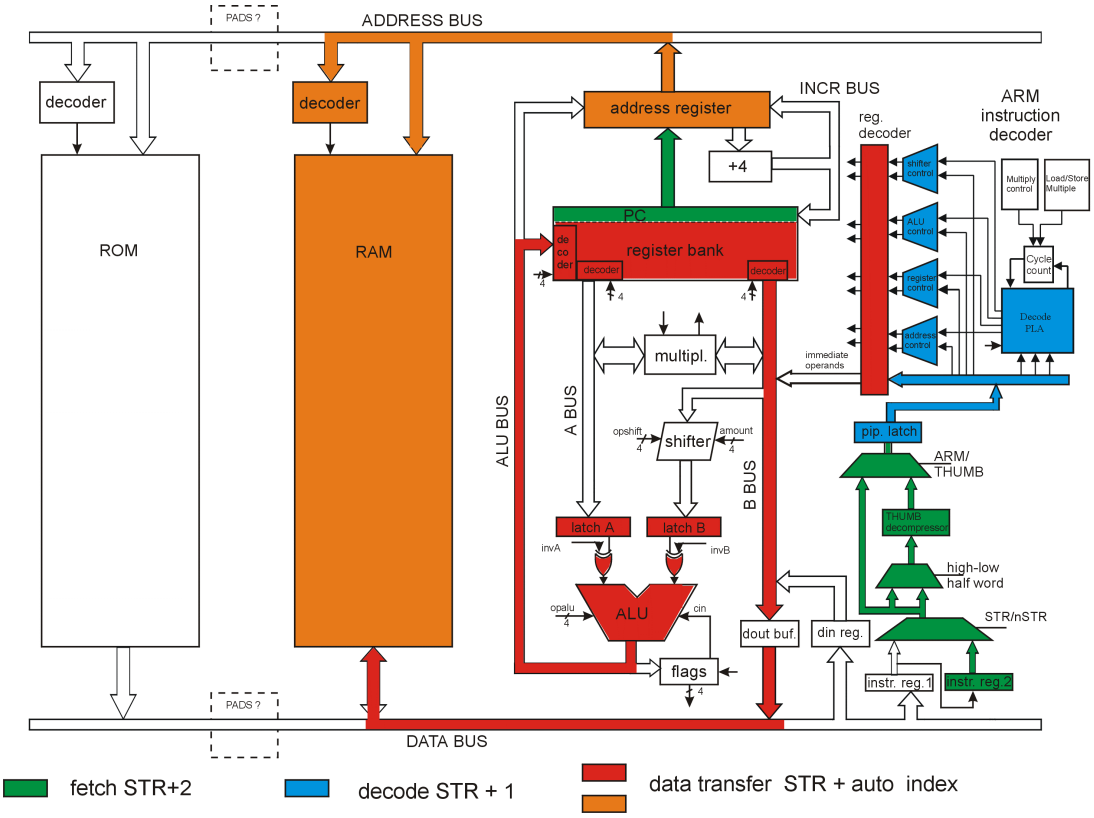


Figure 8: Second cycle of instruction STR execution.

3.4.4 LIST OF RESOURCES ACTIVATED BY AN ELEMENTARY OPERATION

From the previous section it is possible to derive the list of basic operations and the resources activated by each elementary operation.

3.4.4.1 ACTIONS FOR AN ADDRESS CALCULATION (LDR AND STR)

```
Rn_decoder
Rn_register (read)
A_Bus <= Rn
LATCH_A <= A_Bus
If P = 0 (post)
                LATCH_B <= 0
Else (P = 1, pre)
{               If I = 0 (immediate)
                {   Read immediate_operands_register (bits 0..11) (Read)
                    B_Bus <= immediate_operands_register (bits 0..11)
                    OpShift  <= NOP
                    Amount <= 0
```

```
                    }
                    Else (I = 1, register)
                    {     Rm_decoder
                          Rm_register (read)
                          B_Bus <= Rm
                          Shift_control  <=  immediate_operands_register (bits 4..11)
                    }
                    Shifter (0,0)
                    B_Bus_sh <= Shifter
                    LATCH_B <= B_Bus_sh
}
If U = 1 (add)
                    opALU <= add
Else (U = 0, subtract)
                    opALU  <= subtract
ALU (LATCH A ± LATCH B)
LATCH_ALU <= ALU
ALU_Bus <= LATCH_ALU
Address_register <= ALU_Bus
```

3.4.4.2 ACTIONS FOR A DATA TRANSFER (LDR)

```
Address_register (read)
Address_Bus <= Address_register
AMBA_Address_Bus <= Address_Bus
PADS_Address_Bus <= AMBA_Adress_Bus
RAM_decoder <= Address_Bus
RAM (read)
PADS_Data_Bus <= RAM?
AMBA_Data_Bus <= PADS_Data_Bus
DIN_register <= AMBA_Data_Bus
```

From calculate address:

```
LATCH_A <= Rn
LATCH_B <= 0 or <= immediate or <= Rm
If U = 1 (add)
                    opALU <= add
Else (U = 0, subtract)
                    opALU <= subtract
ALU (LATCH A ± LATCH B)
LATCH_ALU <= ALU
ALU_Bus <= LATCH_ALU
Rn_register <= ALU_Bus
```

3.4.4.3 ACTIONS FOR AN INTERNAL CYCLE (MI LDR)

Rd_decoder
DIN_register (read)
B_Bus <= DIN_register
Opshift <= NOP
Amount <= 0
Shifter (0,0)
B_Bus_sh <= Shifter
OpALU <= B
ALU (B)
LATCH_ALU <= ALU
ALU_Bus <= LATCH_ALU
Rd_register <= ALU_Bus

3.4.4.4 ACTIONS FOR A DATA TRANSFER (STR)

Address_register (read)
Address_Bus <= Address_register
AMBA_Address_Bus <= Address_Bus
PADS_Address_Bus <= AMBA_Adress_Bus?
RAM_decoder <= Address_Bus
Rd_decoder
Rd_register (read)
B_Bus <= Rd_register
DOUT_buffer <= B_Bus
Data_Bus <= DOUT_Buffer
PADS_Data_Bus <= Data_Bus?
AMBA_Data_Bus <= PADS_Data_Bus
RAM <= AMBA_Data_Bus

From calculate address:
LATCH_A <= Rn
LATCH_B <= 0 or <= immediate or <= Rm
If U = 1 (add)
opALU <= add
Else (U = 0, subtract)
opALU <= subtract
ALU (LATCH A ± LATCH B)
LATCH_ALU <= ALU
ALU_Bus <= LATCH_ALU
Rn_register <= ALU_Bus

3.4.4.5 FETCH

Address_register (read)
Address_Bus <= Address_register
AMBA_Address_Bus <= Address_Bus
PADS_Address_Bus <= AMBA_Adress_Bus?
ROM_decoder <= Address_Bus
ROM (read)
PADS_Data_Bus <= ROM?
AMBA_Data_Bus <= PADS_Data_Bus
Fetch_register_1 <= AMBA_Data_Bus

In case of use prefetch queue:
Fetch_register_2 <= Fetch_register_1

3.4.4.6 ADDRESS CONTROL (FETCH)

Address_register (read)
+4 <= Address_register
Incrementer_Bus <= +4
PC_register <= Incrementer_Bus

3.4.4.7 DECODE

Decoding of single cycle instructions:
MUX_STR/nSTR_control <= 0/1
Instruction_Bus32 <= MUX_STR/nSTR
MUX_ARM/nTHUMB_control <= 0/1
Decode_latch <= MUX_ARM/nTHUMB
Decode_Bus <= Decode_latch
Decode_PLA <= Decode_Bus
Decode_PLA ()
Decode_Logic <= (Decode_PLA, Decode_Bus)
Counter_Control <= Decode_PLA

Postdecoding of multiple cycle instructions:
Cycle_Counter <= Counter_Control
Decode PLA <= (Decode_Bus, Cycle_Counter)
Decode_Logic <= <= (Decode_PLA, Decode_Bus)

3.4.4.8 EXECUTE

```
Data Processing Instructions:
Execute_register <= out_decoder
opALU <= opcode
Rd_decoder
Rn_decoder
Rn_register (read)
A_Bus <= Rn_register
LATCH_A <= A_Bus
If I = 0 (non immediate)
{   Rm_decoder
Rm_register (read)
B_Bus <= Rm_register
Shift_control
}
Else (I =1, immediate)
{   B_Bus <= immediate (bits 0..7)
Shift_control
}
Shifter (Shift_control)
B_Bus_sh <= Shifter
LATCH_B <= B_Bus_sh
ALU (LATCH_A opALU LATCH_B)

If  (instructions tst, cmp, cmn, teq)
{                   LATCH_ALU <= ALU
                    ALU_Bus <= LATCH_ALU
                    Rd_register <= ALU_Bus
                    If S = 1
                         Set_FLAGS (ALU)
                    Else (S = 0)
                    {}
}
Else (instructions tst, cmp, cmn, teq)
                    Set_FLAGS (ALU)
```

3.4.5 ENERGY CONSUMPTION OF THE PRINCIPAL RESOURCES

For each basic resource an energy model that takes into account the data is generated. As a matter of example the following resources are evaluated here:

3.4.5.1 BUSSES

The ARM7TDMI architecture has several internal busses to interconnect the different blocks in the Data path. The power estimator need to know several parameters in order to perform an accurate prediction of the consumption of these elements:

- Capacitance of each bus line
- Previous state of each bus line

3.4.5.2 REGISTERS WRITE

The ARM7TDMI architecture has several internal registers to store data and control information. These registers have two read ports and 1 write port connected to the internal busses. The PC register has an additional read and write port. The power estimator need to know several parameters in order to perform an accurate prediction of the consumption of these elements:

- Capacitance of each bus line connected to the read ports
- State of each register
- Structure of the data storage element
- New data to be memorized in the registers
- Technology parameter

3.4.5.3 MULTIPLIER

This block is a 32 x 32 bit carry save multiplier. According to the ARM7TDMI datasheet and **FUR96a** it is based in a 32 x 8 multiplier core implementing a Modified Booth algorithm **BOO50a RUB75a MAC61a**. This internal core is looped through a 64-bit carry save register that is used to complete 32 x 32 operations. The multiplier core also implements strategies early termination operation. This mechanism operates in the internal core and thus the early termination signal is precomputed in every 32 x 8 data operation block.

Multiplier operation can operate in two precision modes: 32-bit result and 64-bit result. Each one can be signed or unsigned. According the configuration and the operators, more or less internal cycles are required for the multiplication. The maximum number of internal cycles is six.

Multipliers are energy consumers due to the high internal activity taking place during operation. Apart from this special behavior, their arithmetic operation function is symmetric while the energy consumption function is totally asymmetric **CAL93a**. Depending on the program, multiplier consumption may have an important role in the energy consumption profile **LEE97a**. This is the reason why the energy consumption model of this block

has been developed with certain detail.

The energy model is described at a logic level of abstraction. Full adders are the smallest block. Logic simulation is performed to determine the activity of each block. Models to estimate the amount of charge extracted from the power source are defined in a library. Activation algorithms are used in order to optimize the speed of the simulation.

3.4.5.4 ALU

This block has the following inputs:

- 32 data input bits (connected to the internal A bus in the ARM7TDMI architecture): DA
- 32 data inputs (known as Op2 in the ARM7TDMI architecture).
- 4 operation code bits
- Carry in bit

And the following outputs:

- 32 data output bits (connected to the ALU bus in the ARM7TDMI architecture).
- 4 condition code bits.

For the consumption model we have assumed an ALU organization very similar as the ARM6 ALU organization [**FUR96a**]: a multiplexer in the first operand and a multiplexer and XOR gates in the second operand allow performing forward and reversing subtractions. Also in the ALU, logic and arithmetic operations are executed in different blocks and the results are connected to a multiplexer. Finally, we assume the 32-bit adder is built by using a Carry Select Adder structure [**CAL93b**]. At the ALU data outputs an auxiliary block detects the Zero, Negative, Carry out or Overflow conditions.

The consumption estimator needs to know the previous value of the operands and opcodes to have information on the internal state of the ALU. Then, by knowing the new operand and opcodes, will calculate the activity and thus the consumption of this block.

Parameters to known are the following:

- Previous internal state of the ALU
- Data and opcode inputs (known from the tracer and debugger)
- Details on the organization and structure of the ALU (more information, more accuracy in the estimation)

- Technology parameter
- Capacitance of the ALU bus.

By knowing these parameters, the estimator will calculates the amount of electrical charge drawn from the power supply associated with each change in the nodes of the ALU.

3.4.5.5 BARREL SHIFTER

This block has the following inputs:

- 32 data input bits: DI (connected to the internal B bus in the ARM7TDMI architecture [**ATM99a**])
- 1 carry input bit: CI
- Shift amount: SA (5 bits in the ARM7TDMI architecture).
- Shift operation: SO (4 bits in the ARM7TDMI architecture).

And the following outputs:

- 32 data output bits: DO (connected to ALU in the ARM7TDMI architecture)
- 1 carry output bit: CO

The internal structure of this block is very regular and allows building a consumption model based on the following parameters:

- The change in the DI bits (known from the tracer and debugger)
- The SA value (known from the tracer and debugger)
- The SO code (known from the tracer and debugger)
- Technological parameters.
- Structural model of the barrel shifter (BS).

The consumption model will assume a standard structure for the barrel shifter [**FUR96a**]. The model will need also as input the state of the internal nodes of the BS. This information is obtained from the previous values of the DI bits (B bus in the ARM7TDMI architecture), and SA and SO bits. The nodes of the BS will be weighted depending on their function (control or data).

Each change on any internal node is associated to a given amount of charge drawn from the power supply. By adding the contribution of the transition in each node the total consumption of the block may be computed [**CHA92b**].

3.4.5.6 DECODER

The ARM7TDMI decoder has three structural components, which relate to each other.

- An instruction decoder PLA. This unit uses some of the instruction bits and an internal cycle counter to define the class of operation to be performed on the data-path in the next cycle.
- Distributed secondary control circuits associated to each of the major data-path function blocks. This logic uses the class information from de main decoder PLA to select other instruction bits and/or processor state information to control the data-path.
- Decentralized control units for specific instructions that take a variable number of cycles to complete (load and store multiple, multiply and coprocessor instructions). Here the main decoder PLA locks in fixed state until the remote control unit indicates completion.

A 'cycle-count' block distinguishes the different cycles in multi-cycle instructions so that the decoder PLA can generate different control outputs for each cycle.

3.4.5.7 ADJUST TECHNIQUES WITH THE HELP OF EXPERIMENTAL RESULTS

Each operation contains a reduced set of parameter to evaluate its implication on the instruction power consumption. These parameters are sensitive to voltage and technology size. The value of each parameter can be obtained by the adequate instrumentation of a set of a set of specific instruction sequence from which the parameters can be evaluated.

3.4.6 SUMMARY

This third evaluating method takes into consideration aspects of the processor architecture as well as the data transferred or processed in each cycle. A computing tool that includes a tracer of the instruction process is able to compute the energy consumed by a set of instructions from its cycles and the use of resources. This technique is able, through the energy evaluation equations in each resource to predict the effect on consumption of changes of voltage and technology.

4 RESULTS AND EXPERIMENTATION

4.1 INTRODUCTION

In this chapter, after a brief description of the different ARM tools available, we first present the measuring system used to obtain all the experimental data on the consumption of the ARM7TDMI microcontroller. Then, we describe some results from the Simple Bus Cycle Energy Estimator and a short explanation of the pipeline reconstruction with some results of the Enhanced Cycle Energy Estimator.

4.2 ARM TOOLS

4.2.1 ARMULATOR & TRACER

Since the aim of this project is to handle with an ARM7TMDI processor, it seems logical that the most appropriate thing would be using as much as possible the available ARM tools. There is a collection of tools and applications available in the *ARM Software Development Toolkit (SDT)* suite, which makes possible writing and debugging applications for the ARM family of RISC processors, and which are really useful to know the behavior of the routine written.

Having a program executable image it is possible to follow cycle-by-cycle the program instruction execution. First of all, however, we must have obtained the instruction-by-instruction execution trace taking advantage of one of those applications, the Tracer. To do so, we will have previously used such an ARM development tools, via command-line or via Windows, in order to get a correctly compiled and debugged executable image from its source code. Next, by means of the ARMulator and the Tracer, two of the utilities in the supporting software included in the suite, we will be able of obtaining the trace generated when the executable image is executed.

The ARMulator is a program which emulates the instruction sets and architecture of various ARM processors providing an environment for the development of ARM-targeted software on our workstation or PC, and it will allow to run different applications (either in ARM, THUMB, C, C++…) in a hardware-independent way. Because it is instruction-accurate, it models the instruction set and counts cycles accurately. Despite so, it is not a cycle-accurate model because it does not model the precise timing characteristics of processors. This emulator is transparently connected to the ARM debuggers (either command–line or Windows debuggers) to provide, as it works with independent self-contained C models which communicate with the ARMulator through a set of defined interfaces, a hardware-independent ARM software development environment. It also allows adding extra models, without altering the existing ones, which will interface to the ARMulator's external interface. Communication between debugger and ARMulator takes place across the Remote Debug Interface (RDI), the same as in case of doing the development on a hardware stage, but having the models instead of the hardware.

One of those models provided is the Tracer; this module, once it is properly configured, will record memory accesses, instruction executions and events to an RDI log window or file and, so, we will obtain though the ARMulator the program execution trace. Alternatively, we could also link our own tracing code onto the Tracer module, allowing real-time tracing.

Finally, once we have obtained the trace of the routine under test, with our dedicated software, we will process all of this information to achieve our goal, which is to estimate the consumption evolution of our program.

4.2.2 TRACE FORMAT

A file generated by the Tracer will be the starting point for our tool; this file will contain the execution trace of our program. In a trace file, there are three types of line:

- Trace memory lines (M lines)
- Trace instruction lines (I line)
- Trace event lines (E lines).

The format of the trace memory (M) lines is as follows: *access address data*

Where 'access' contains the following information: indicates the type of memory cycle (sequential, non-sequential, idle, coprocessor); a read or a write operation; the size of the memory accesses (word (32 bits), halfword (16 bits), byte (8 bits)); if the memory access is an opcode fetch, etc.

The format of the trace instruction (I) lines is as follows: *[IT | IS] instr_addr opcode disassembly*

Where IT means instruction taken and IS instruction skipped.

Example de trace generated by the ARMulator Tracer:

```
Date: Mon Oct 23 18:07:56 2000
Source: Armul
Options: Trace Instructions  (Disassemble) Trace Memory Cycles  (Idles)

MNR4O__ 00008000 E59F1024
MSR4O__ 00008004 E59F0024
MSR4O__ 00008008 EB000002
IT 00008000 e59f1024 LDR r1, 0x802c
MNR4___ 0000802C 00008038
MI
```

```
MSR4O__ 0000800C E3A00018
IT 00008004 e59f0024 LDR      r0,0x8030
MNR4___ 00008030 0000804E
MI
MSR4O__ 00008010 E59F101C
IT 00008008 eb000002 BL 0x8018
MNR4O__ 00008018 E4D12001
         .
         .
         .
MSR4O__ 00008018 E4D12001
IT 00008010 e59f101c LDR r1, 0x8034
MNR4___ 00008034 00020026
MI
MSR4O__ 0000801C E4C02001
IT 00008014 ef123456 SWI 0x123456    …
```

Thus from this trace we will obtain the information which will allow to know the temporary evolution of the program, since we will know the temporary sequence of all memory accesses (memory accesses and fetches) and instruction executions. Processing in an appropriate way this information will allow reconstructing the cycle-by-cycle pipeline temporary evolution, as we will be able to know when and what fetch is being generated (knowing, moreover, attributes like access address, instruction which is being fetched, etc.); when and what instruction is being decoded; and when and what instruction is being executed (with its respective read/write memory accesses characterized, and/or its respective internal execution cycles, in case they exist).

4.3 MEASURING SYSTEM

4.3.1 COMPONENTS OF THE MEASURING SYSTEM

The measuring system is shown in Figure 9. The elements of this system are the following:

1. AT91EB01 Evaluation Board from ATMEL.
2. Logic Analyzer TLA613 from TEKTRONIX.
3. Digital Storage Oscilloscope TDS744A from TEKTRONIX.
4. Differential Probe P6248 from TEKTRONIX.
5. PC with LABVIEW, MATLAB and dedicated software.

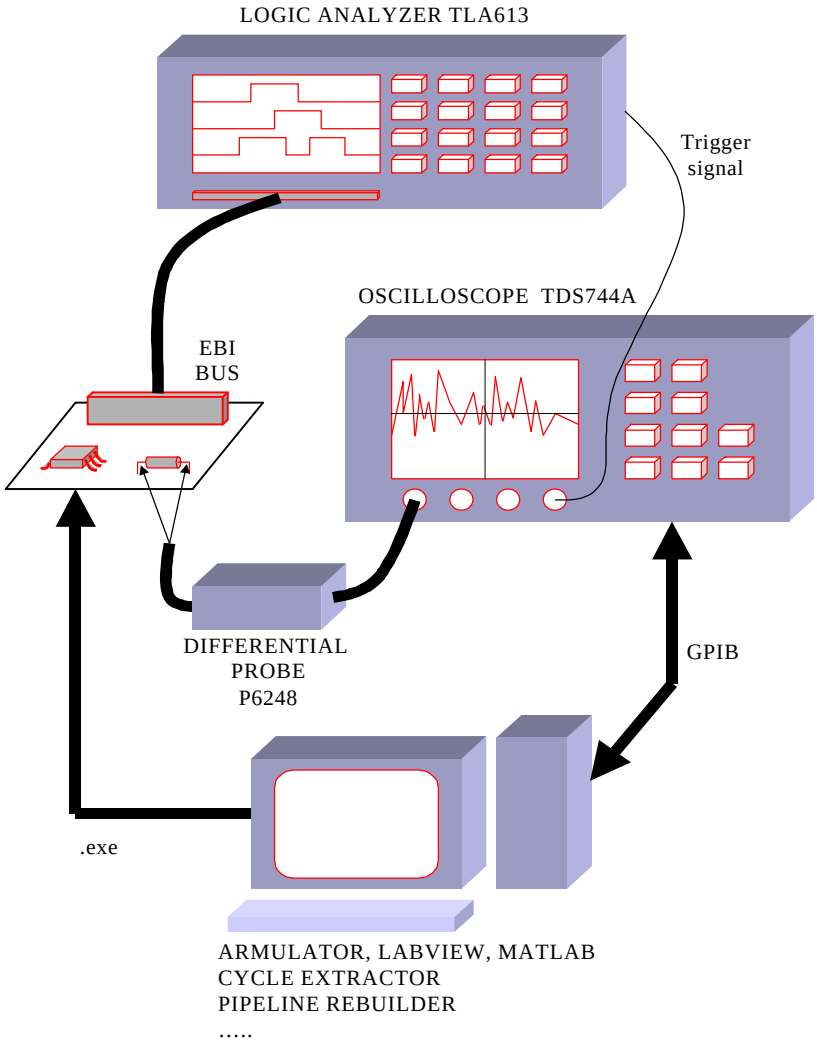


Figure 9: Block diagram of the measuring system.

An image of the measuring system setup is shown in Figure 10:

Figure 10: Measuring system setup.

4.3.1.1 AT91EB01 EVALUATION BOARD

A block diagram of the AT91EB01 Evaluation board [**ATM00**] is shown Figure 11. The picture of the Evaluation Board is shown in Figure 12. This board consists of an AT91R40807 microcontroller together several peripherals:

- Two serial ports
- Reset button
- Three applicative buttons
- Three LED's
- 512 Kbytes 16 bit SRAM
- 128 Kbytes 16 bit Flash
- 20 pin JTAG interface connector
- A removable jumper connecting the board power supply pins (3.3 volts) to the power pin of the AT91R40807 microcontroller.
- An EBI Bus connector
- An I/O connector.

There are several debugging systems available for the AT91EB01 board: ARMULATOR, EmbeddedICE, Multi-ICE and Angel debug Monitor. During the development of the VIP project only the ARMULATOR tool has been used. To measure the consumption of the AT91R40807 microcontroller, the removable jumper has been eliminated and replaced by a small SMD resistor.

The AT91R40807 microcontroller integrates an ARM7TDMI with its embedded ICE interface, memories and peripherals. Its architecture consists on two main busses, the Advanced System Bus (ASB) and the Advanced Peripheral Bus (APB). The ASB interfaces the ARM7TDMI processor with the on-chip 32-bit memories, the External Bus Interface (EBI) and the AMBA Bridge. The AMBA Bridge drives the APB, which is designed for accesses to on-chip peripherals.

The AT91R40807 microcontroller embed 8 Kbytes of SRAM and 128 Kbytes of ROM. These internal memories are directly connected to the 32-bit data bus and are single-cycle accessible. Also the AT91R40807 microcontroller feature an External Bus Interface (EBI), which enable connection of external memories and application-specific peripherals.

The AT91R40807 microcontroller integrate several peripherals which are 32-bit accessible by the AMBA Bridge. These peripherals are:

- Chip Select Module.
- Power Saving Module.
- Advanced Interrupt Controller.
- Parallel Input-Output Controller.
- Watch Dog Module.
- Special Functions Module.
- Two USARTs.
- Three-channel 16-bit Timer-Counter.

As summary, the AT91EB01 Board has its architecture properly fitted to emulate the final architecture of the microcontroller to be designed in the VIP project. Also, taken into account the debug tools, it is well performed to run any ARM7TDMI program and it is, with minor changes, well prepared for the measurement of the power/energy consumed by the ARM7TDMI processor.

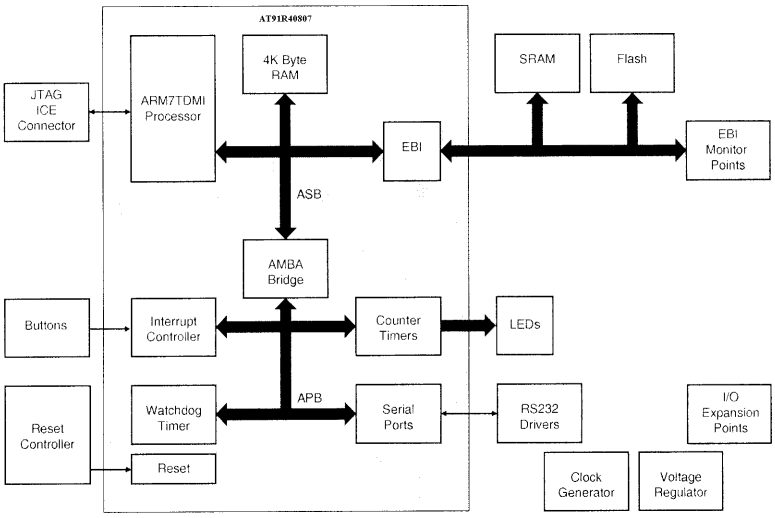


Figure 11: Block diagram of the AT91EB01 board

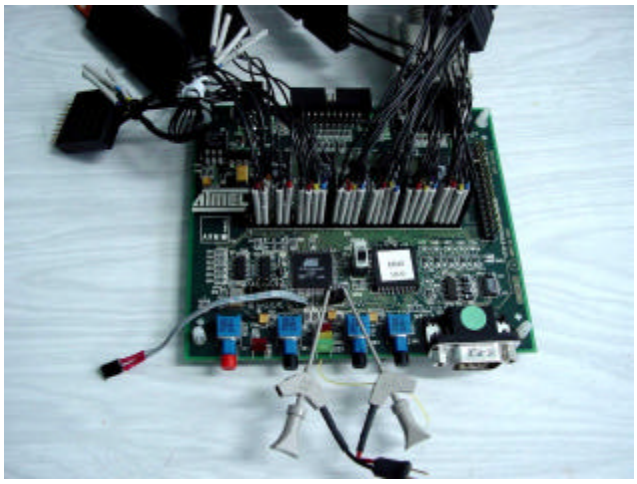


Figure 12: The AT91EB01 evaluation board with the logic analyzer probes.

4.3.1.2 P6284 DIFFERENTIAL PROBE [**TEK00A**]

To perform an accurate measurement of the power/energy consumed by the ARM7TDMI processor, we need a probe with high bandwidth, in order to disturb at minimum the waveform captured by the probe. Also, a differential probe is needed to perform the measurement using the method outlined below.

4.3.1.3 TDS744A DIGITAL STORAGE OSCILLOSCOPE

For the current measurement, a DSO is needed with enough bandwidth, memory and speed to the current waveform without loss of important information. The TDS744A DSO [**TEK00**] is a valuable tool which have the speed, memory and performances suitable for the requirements of the VIP project. An image of the DSO and the differential probe is shown in Figure 13.

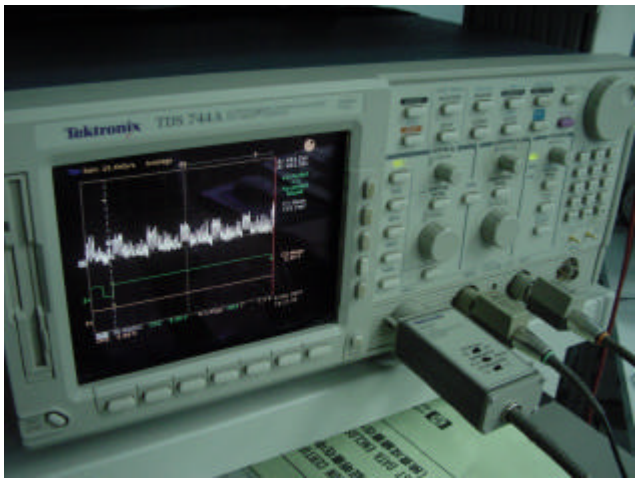


Figure 13: TDS744A DSO and P6248 differential probe.

4.3.1.4 TLA613 LOGIC ANALYZER

This instrument [**TEK00**] has software specialized prepared to rebuild the cycle activity of the ARM7TDMI processor from the events it captures from the EBI BUS. Its 132 input channels are enough to connect all EBI Bus signal plus some other external signals that allow to synchronize the Evaluation Board with the measuring system.

4.3.1.5 ARMULATOR, MATLAB, LABVIEW AND DEDICATED SOFTWARE

The controller PC includes software tools that process the information supplied by the DSO. This software tools include standard tools, like ARMULATOR or MATLAB command files, and dedicated tools devoted to extract and present the results.

- ARMULATOR [**ARM00**]: a brief summary of the capabilities of this simulator is presented in point 3.2 of this document.
- MATLAB [**MAT99**]: it is a well known mathematical tool that have the data processing capabilities needed for the present application.
- LABVIEW [**LAB98**]: it is also a well-known standard tool able to control the communication between the DSO, the controlling PC and the Logic Analyzer.

Dedicated Software: CYCLEPROCESS and REGRESS tools summarized in 3.2.

4.3.2 MEASUREMENT METHOD

The procedure to measure the energy consumed by the ARM7TDMI when it is running a given program is the following:

a. There is connected a small SMD resistor (5.28 ohm) in series with the power supply pins of the ARM AT91R40807 chip inside the Evaluation Board (Figure 9). Thus, all current the current supplied by the power supply to the ARM chip passes through this resistor and develops a drop voltage that can be measured by the oscilloscope. The differential probe P6248 is connected in parallel with the sensing resistor. This probe has a bandwidth of 1.5 GHz, enough to capture without appreciable distortion the waveform of this drop voltage. The method to measure the consumption of a digital circuit by means of the measurement of drop voltage in a sensing resistor has been widely used and is analyzed in [**RIU00**].

b. In order to synchronize the triggering of the oscilloscope with the start of the program, the Logic Analyzer is connected to the EBI Bus of the Evaluation Board and it is continuously observing the address and the instruction that is executed in each clock cycle in the microcontroller. When the current address and instruction is the first instruction of the program to be analyzed, the Logic Analyzer triggers the oscilloscope activating an auxiliary output (Figure 9).

c. On the other hand, we know the number of clock cycles the program need to be completed, then we

adjust the horizontal sweep of the oscilloscope properly, thus storing in it the whole current waveform. After this action, the oscilloscope sends the whole current waveform to the controlling computer to be processed later. Also, the oscilloscope sends the mean of the drop voltage in the sensing resistor and the mean of the power supply voltage.

The LABVIEW software running in the computer executes the previous there steps procedure twenty times and, in order to obtain a reliable result, calculates the actual value of the drop voltage in the resistor and the actual value of the power supply voltage by averaging these twenty samples.

In this way, by processing the data captured by the measuring system we can obtain the following data referred to the period of time a given program runs in the Evaluation Board:

a. The average power consumed by the ARM while the program is running.
b. The energy consumed by the ARM in the period of time the program is running.
c. The waveform of the instantaneous power consumed by the ARM during the execution of the program.

This information will be processed later by the REGRESS tools in order to obtain or adjust the parameters of the consumption estimators.

In the figures Figure 14 and Figure 15 two examples of power waveforms can be seen. The first one shows a piece of the consumption waveform of the ARM7TDMI when it is executing a program that calculates the Beta function (a mathematical function from a library). The second shows the waveform when the processor is executing a Load/Store routine.

To check the reliability of the values obtained from the measurement system, we have repeat many times the same measurement for a several programs. To illustrate the results, in Figure 16 it is shown the histogram of the values of 28000 measurements of the energy consumed during the execution of a single program. As can be seen, the dispersion is very small obtaining values of standard deviation over the mean value lower than 0.003, that is to say, lesser than 0.3 %. Moreover, the form of the histogram reveals a bell-shaped curve, as is expected.

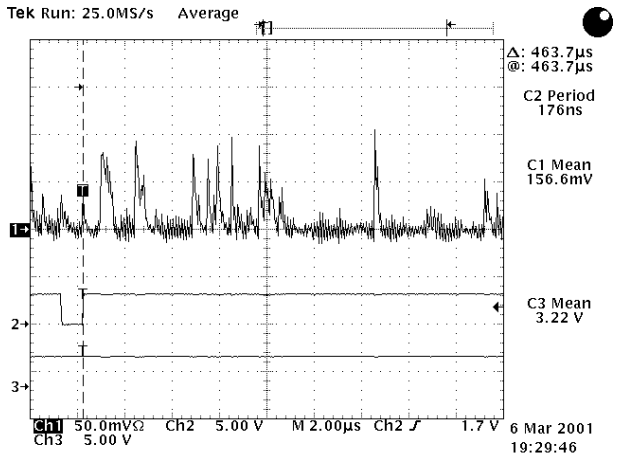


Figure 14: Waveform of the ARM7TDMI consumption when it is calculating a Beta function.

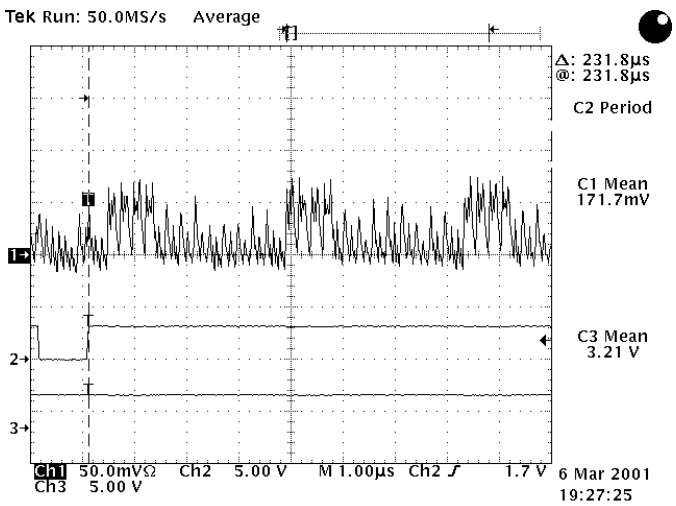


Figure 15: Waveform of the ARM7TDMI consumption when it is executing a LOAD /STORE program.

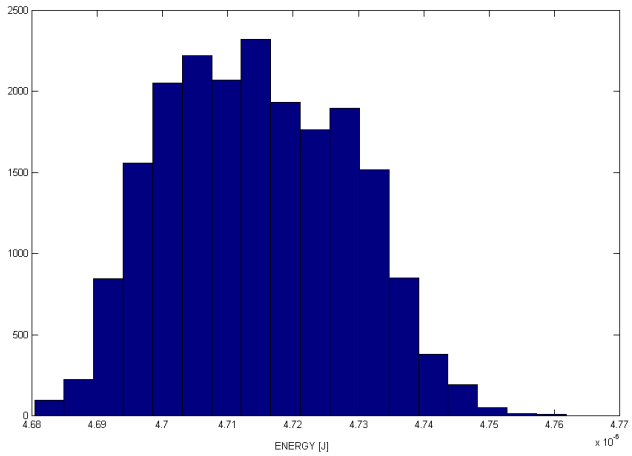


Figure 16: Histogram of 28000 energy measurements of the energy consumed by the ARM7TDMI during the execution of the same program.

4.4 RESULTS

4.4.1 INTRODUCTION

We have performed measurements to compare the estimated and real values of the power/energy consumed by the ARM7TDMI. Actually this comparison has been done only for the *Simple Bus Cycle Energy Model*. Thus, the rest of this section is referred to this model.

In order to compare the quality of the results using the *Simple Bus Cycle Energy Model*, we have performed

preliminary estimations using the most simple energy estimator conceivable: the counting of the number of clock cycles a program need to be executed. After the measurement of 18 small ARM programs running in external memory and 10 of these ARM programs running in internal memory we have obtained the results shown in Figure 17.

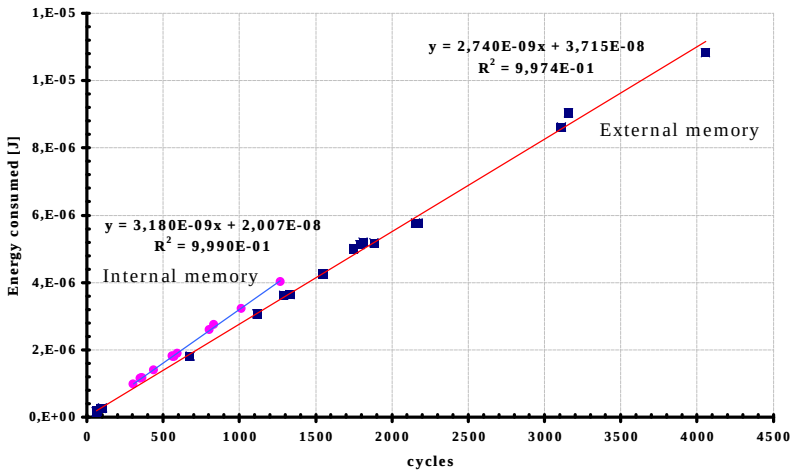


Figure 17: Energy consumed for the ARM7TDMI during the execution of several small programs.

As can be seen, the correlation between the number of clock cycles and the energy consumed is quite good. In this way, a "dumb" estimator would be to obtain the energy by cycle and multiply the number of cycles the program spends by the energy/cycle coefficient. In spite to its simplicity, this simple metric to estimate the consumption suffers from several problems. At first, any information about the temporal evolution of the energy consumed is lost because all information we have is the average consumption per cycle. Secondly, the precision of the estimation is quite good but can be improved, as will be shown later.

Our Simple Bus Cycle Energy Estimator improves the estimation obtained by the "dumb" estimator in both fields: improving the precision and supplying information about the time evolution of the instantaneous power consumption in a cycle-by-cycle basis.

4.4.2 SIMPLE BUS CYCLE ENERGY ESTIMATOR

As was explained in the point 3.2 of this document, the *Simple Bus Cycle Energy Model* estimates the energy consumed by the ARM processor by running a training set of programs, then by measuring the consumed energy for each program, and finally obtaining a set of coefficients associated with the basic types of bus cycles. These actions have been made for a set of small benchmark programs and a set of coefficients has been obtained. In Figure 18 we show a comparison between the estimated (X-axis) and real (Y-axis) consumption for this small set of programs is shown. As is expected, the concordance is very good.

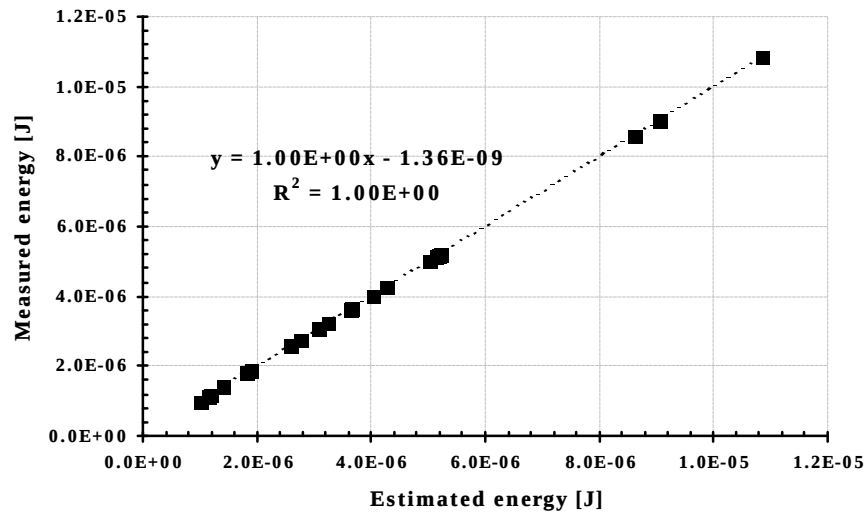


Figure 18: Correlation between the energy measured and estimated by using the Simple Bus Cycle Energy Estimator on a small set of programs.

Each one of the coefficients obtained is an estimation of the energy consumed by one type of bus cycle executed by the microprocessor. After the coefficients were obtained, their reliability was tested estimating from them the consumption of a set of programs with larger size. The agreement between the estimated and the measured energy for this second set of programs was very good, as can be seen in Figure 19. The four isolated points on the straight line are the estimated and measured energies of four large programs. The points in the bottom left corner of Figure 19 are from the training set of programs.

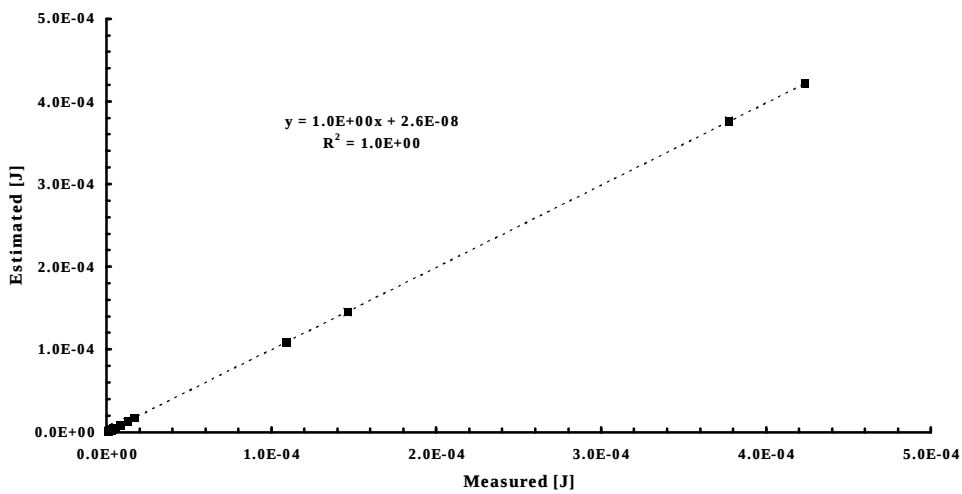


Figure 19 : Test of the Simple Bus Cycle Energy Estimator with some programs.

As we can see, the agreement is very good and shows the accurate estimation we can obtain using this simple model.

4.4.2.1 RELIABILITY OF THE COEFFICIENTS

The previous results were obtained by using the procedure described in 3.2. In order to check the reliability of the values of these coefficients, we have included in the REGRESS tool the calculation of the confidence intervals of these values.. The best results obtained are shown in Figure 20, where the circles are the values of the coefficients of three types of Bus Cycles and the 95 % confidence intervals for 36 and 46 training programs are also shown. These coefficients are related with the energy consumed during the execution of a FETCH, LOAD or STORE cycles of programs running in external memory.

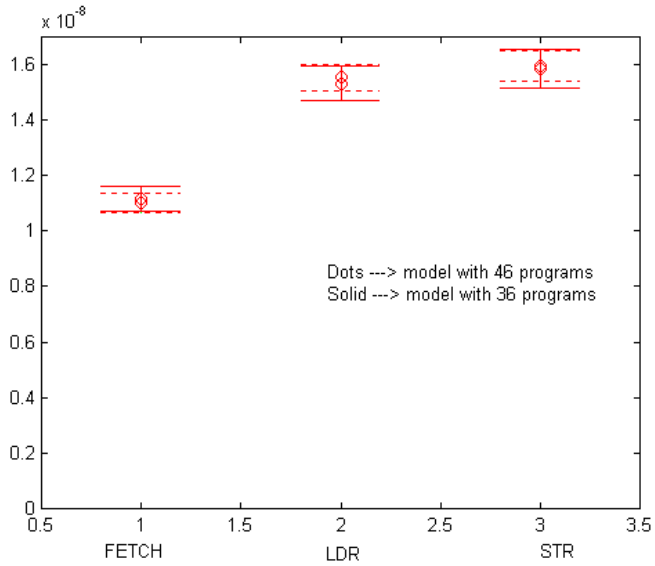


Figure 20: 95 % confidence intervals for the coefficients of FETCH, LDR and STR cycles.

4.4.2.2 POWER CHRONOGRAMS

Also, from the *Simple Bus Cycle Energy Model* is possible to obtain a chronogram of the power consumed by the ARM processor in a cycle-by-cycle basis. This chronogram can be compared with the real waveform of the ARM consumption. An example is shown in Figure 21. In this figure, we show a small piece of the power waveform of the ARM7 when it is running a program that executes many LDR a STR cycles. The X-axis is the time in arbitrary units and the Y-axis is Power in Watts. As can be seen, there is a good concordance between the estimated and the real evolution of the power.

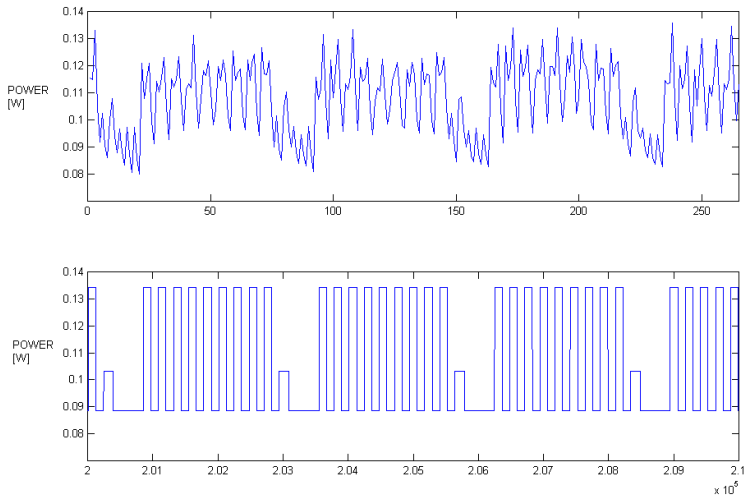


Figure 21 : Power chronogram of a program. From the oscilloscope trace (top), from the estimator (bottom).

4.4.2.3 SUMMARY

As is shown, by using the *Simple Bus Cycle Energy Model* we obtain gives a good estimation of the power/energy consumption of the ARM7TDMI microcontroller, both in average and as a time function. The results obtained are referred to a particular implementation of the microcontroller. However, there is some features of this model that are interesting from a general point of view.

- The methodology used is completely general.
- For programs running in internal memory, the consumption estimated is the core consumption (no pads activity).
- The results obtained for the particular ARM7TDMI (0.35 µm @ 3.3 volts @ 32 MHz) measured, are starting points to be scaled to other technologies or power supply voltages.
- Allow us to build a workbench to test and compare more accurate estimators like the ones we are developing in the VIP project.

4.4.3 ENHANCED CYCLE ENERGY ESTIMATOR

4.4.3.1 PIPELINE RECONSTRUCTION TOOL

In the next lines, it is proposed a system to know the core activity during the program execution, which later will allow us to connect the consumption with the activity generated in the core. Our starting point will be the trace provided by the ARMulator's Tracer. Although, at first, this trace can be obtained running manually the ARMulator's Tracer for the program to be traced (which can be written in assembler, C, Pascal or F77 language)

from one of the ARM debuggers (either command–line or Windows debuggers), later on, it could be included inside the application, being an automated process which would start from our application.

As we have already explained, to know the core activity, a complex model, including a cycle-by-cycle pipeline activity reconstruction, will be used. The information generated by the tracer will allow us to know the pipeline and program temporary evolution, since it provides information about all the memory accesses (and, thus, also all the fetches) and the instruction executions in a sequential way. In other words, as we have already said, we are able of knowing when and what fetch is being generated, when and what instruction is being decoded; and when and what instruction is being executed, so, our aim will be processing all of this information in an appropriate way and reconstructing the pipeline cycle-by-cycle evolution.

To reconstruct the pipeline we will use different C routines in our application, which will read the trace file, extracting, and, afterwards, processing the information cycle-by-cycle and reconstructing the pipeline. These routines, bearing in mind that the ARM7 processor has a three-level pipeline, will keep a record of the last instructions fetched in the last cycles. Therefore, knowing the instruction which is being executed at the present moment and the ones which have been executed (in case they exist) during the previous cycles, it will be able of deducing when the decoding of the present instruction executed happens (its fetch and execution will be already known just reading the trace, so it will not be necessary to deduce them). Using this method, we will have available all the instruction fetches and instruction executions during the cycle where it happens, but we will not know when an instruction decoding has taken place until this instruction has been executed.

For instance, if the trace were something like:

```
                .
                .
                .
MSR4O__ 00008010    E3A0200A
IT      00008008    e3a05000        MOV r5, #0
MSR4O__ 00008014    E4903004
IT      0000800C    e3a06000        MOV r6, #0
MSR4O__ 00008018    E4904004
IT      00008010    e3a0200a        MOV r2, #0xa
MSR4O__ 0000801C    E0F65493
IT      00008014    e4903004        LDR r3, [r0], #4
MNR4___ 00008048    00000002
MI
MSR4O__ 00008020    E4815004
IT      00008018    e4904004        LDR r4, [r0], #4
MNR4___ 0000804C    00000002
     .
     .
     .
```

After reading the first line we would know that a fetch at the address 8010 of the instruction E3A0200A has taken place, so our pipeline situation would be:

Cycle i	FETCH: E3A0200A		
Cycle i+1			

When reading the second line, we will know that the instruction E3A05000, at the address 8008, is being executed:

Cycle i	FETCH: E3A0200A		EXEC: E3A05000
Cycle i+1			

The next line in the trace, the third one, will inform us that another fetch, of the instruction E4903004, at the address 8014 happens; and in the fourth line we have an execution, now for the instruction E3A06000, at the address 800C; adding this information to our pipeline:

Cycle i	FETCH: E3A0200A		EXEC: E3A05000
Cycle i+1	FETCH: E4903004		EXEC: E3A06000
Cycle i+2			

But to execute this instruction in this cycle, it must have been decoded in the previous one, therefore we will already know when the decoding happened:

Cycle i	FETCH: E3A0200A	DEC: E3A06000	EXEC: E3A05000
Cycle i+1	FETCH: E4903004		EXEC: E3A06000
Cycle i+2			

Reading the fifth line in the trace, we will obtain a new fetch; now we are fetching the instruction E4904004 at the address 8018. The next reading provides information about the next action at the processor, a new execution, of the instruction E3A0200A, which is at the 8010 position. Therefore we will deduce that the decoding happened in the previous cycle. Adding all of this information in our pipeline table:

Cycle i	FETCH: E3A0200A	DEC: E3A06000	EXEC: E3A05000
Cycle i+1	FETCH: E4903004	DEC: E3A0200A	EXEC: E3A06000
Cycle i+2	FETCH: E4904004		EXEC: E3A0200A
Cycle i+3			

…

In this example we only had single-cycle instructions, but we could also have had multi-cycle instructions where read/write accesses or internal cycles could have taken place and would have been considered as a part of the instruction execution. In such a case the method would be the same, as we can see if we complete the few lines of our example trace:

...

Cycle i	FETCH: E3A0200A	DEC: E3A06000	EXEC: E3A05000
Cycle i+1	FETCH: E4903004	DEC: E3A0200A	EXEC: E3A06000
Cycle i+2	FETCH: E4904004	DEC: E4903004	EXEC: E3A0200A
Cycle i+3	FETCH: E0F65493		EXEC: E4903004
Cycle i+4			MNR4 8048
Cycle i+5		DEC: E4904004	MI
Cycle i+6	FETCH: E4815004		EXEC: E4904004
Cycle i+7			MNR4 804C

...

Example of a reconstructed pipeline:

```
Fetch Ac.Fetch Ad.Fetch Dt..Decod Op..Mem Acc.M Acc Ad.Mem Data..Inst Opc.Inst Add.Inst Disassembly
        |       0|       0||       0||        |       0|       0||       0|     0|
        |       0|       0||       0||        |       0|       0||       0|     0|
        |       0|       0||       0||        |       0|       0||       0|     0|
 MNR4O__|    8000|E59F1024||       0||        |       0|       0||       0|     0|
 MSR4O__|    8004|E59F0024||E59F1024||        |       0|       0||       0|     0|
 MSR4O__|    8008|EB000002||       0||        |       0|       0||E59F1024|  8000| LDR r1,0x802c
        |       0|       0||       0||MNR4___|    802C|    8038||       0|     0|
        |       0|       0||E59F0024||MI     |       0|       0||       0|     0|
 MSR4O__|    800C|E3A00018||       0||        |       0|       0||E59F0024|  8004| LDR r0,0x8030
        |       0|       0||       0||MNR4___|    8030|    804E||       0|     0|
        |       0|       0||EB000002||MI     |       0|       0||       0|     0|
 MSR4O__|    8010|E59F101C||       0||        |       0|       0||EB000002|  8008| BL 0x8018
 MNR4O__|    8018|E4D12001||       0||        |       0|       0||       0|     0|
 MSR4O__|    801C|E4C02001||E4D12001||        |       0|       0||       0|     0|
                                     .
                                     .
                                     .
 MSR4O__|    8018|E4D12001||       0||        |       0|       0||E59F101C|  8010| LDR r1,0x8034
        |       0|       0||       0||MNR4___|    8034|   20026||       0|     0|
        |       0|       0||EF123456||MI     |       0|       0||       0|     0|
 MSR4O__|    801C|E4C02001||       0||        |       0|       0||EF123456|  8014| SWI 0x123456
```

At the same time we do the reconstruction, and, as we know what resources are being used in each fetch, decoding and execution, we will be able of estimating the cycle consumption. To do so, we will use some C routines written for this purpose; those routines will be called once each cycle is completed and will calculate the consumption according the resources and data used. Keeping the record of each cycle consumption will have the

cycle-by-cycle consumption evolution.

Once the pipeline has been reconstructed, or even while we are doing the reconstruction, we can obtain all the information we are interested in, codifying with a bit all the actions which take place during the clock cycle and, thus, in this way, characterizing the cycles. This will allow us to extract not only all the codified information about what it is happening in the cycle, but also how many times each action took place during the program execution and how and what different cycles where executed in the program.

An example of cycle information extracted would be:

F || DEC || RI1 | RI2 | RI4 || WI1 | WI2 | WI4 || RE1 | RE2 | RE4 || WE1 | WE2 | WE4 || MIL | MIM | MIO || DAT_PRC_INST | MULT_INST | MOV_INST | B_EXCH_INST | LD/ST_S_INST | SWAP_INST | LD/ST_M_INST | BRC_INST | INT_INST

1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0
0 || 0 || 0 | 0 | 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
0 || 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 1 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0
0 || 0 || 0 | 0 | 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
0 || 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 1 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0
1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0
0 || 0 || 1 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
0 || 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 1 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0

.
.
.

1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0
0 || 0 || 0 | 0 | 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
0 || 1 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 1 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
1 || 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 || 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1

fetch:143 dec:0
int_rm1:22 int_rm2:0 int_rm4:3 int_wm1:22 int_wm2:0 int_wm4:0
ext_rm1:0 ext_rm2:0 ext_rm4:0 ext_wm1:0 ext_wm2:0 ext_wm4:0
mil:25 mim:0 mio:0
datapr:22 mult:0 move:2 exch:0 stlo_s:47
swap:0 stlo_m:0 brch:22 intrr:1

On the other hand, this codified information will be used to differentiate the cycles executed, which will also be counted. Therefore, knowing the different kind of cycles that we can find in the program and how many times each one happened during the program execution, we would know the vector where the times that each cycle happened are the components. Repeating this operation for many programs would allow us to have the cycle matrix according to the different type of cycle executed for a program or a cycle matrix according to the

number of times that each cycle coefficient has happened. Moreover, measuring the consumption for each program will provide us the consumption vector used in the multiple regression. As we have already explained, this matrix will be the one used to calculate the coefficients with the multiple linear regressions. The routines to do all of these actions will be implemented, as usual, in C.

Example of the different cycle executed in a program:

```
F || DEC || INT_MEM_ACC | EXT_MEM_ACC || MIM | MIO || DAT_PRC_INST | MULT_INST | MOV_INST |
B_EXCH_INST | LD/ST_S_INST | SWAP_INST | LD/ST_M_INST | BRC_INST | INT_INST ||  TIMES

CYCLE TYPES:
1 || 0 || 0 | 0 || 0 | 0 || 0 | 0 | 0 | 0 | 0| 0 | 0 | 0 | 0 ||   25
1 || 1 || 0 | 0 || 0 | 0 || 0 | 0 | 0 | 0 | 0| 0 | 0 | 0 | 0 ||   24
1 || 0 || 0 | 0 || 0 | 0 || 0 | 0 | 0 | 0 | 1| 0 | 0 | 0 | 0 ||   46
              .
              .
              .
1 || 0 || 0 | 0 || 0 | 0 || 0 | 0 | 0 | 0 | 0| 0 | 0 | 1 | 0 ||   21
0 || 1 || 1 | 0 || 0 | 0 || 0 | 0 | 0 | 0 | 0| 0 | 0 | 0 | 0 ||   21
1 || 1 || 0 | 0 || 0 | 0 || 1 | 0 | 0 | 0 | 0| 0 | 0 | 0 | 0 ||   20

TOTAL CYCLES: 215
```

4.4.3.2 SUMMARY

Using the Enhanced Cycle Energy Estimator allows reconstructing the pipeline evolution for a program, therefore, knowing which is the consumption of every block in the microprocessor activated by any of the actions taking place in the pipeline, the Tool would provide the evolution of the consumption all through the time.

As it is shown, the methodology used in the previous section is a completely general methodology, so we can apply the method to estimate a first value for any of the two approaches presented in the Enhanced Cycle Energy Estimator. In any case, this method would also be useful to tune the final theoretical coefficients for each block in the microprocessor.

As in the section 4.4.2, the results obtained for a particular ARM7TDMI (0.35 μm @ 3.3 volts @ 32 MHz) would be starting points to be scaled to other technologies or power supply voltages.

5 CONCLUSIONS

Microprocessor blocks use to be part significant of System on Chip applications. In most of these applications the power consumption prediction at the design stages of the system is a major concern. The consumption of a moderate size digital circuit can be evaluated with adequate accuracy if details of physical implementation are known. But microprocessors are large and complex systems, moreover not always the physical details are known. Together with this fact at the different levels of the synthesis process designers exhibits different ratio requirements of speed/accuracy on the prediction.

The responsibility of UPC in this task is to analyze the state of the art of the power evaluation techniques and tools, introduce innovative results and generate a Tool the MicroPP software designers could use to evaluate the consumption of the ARM and OAK (TEAK) processors and optimize software or use the results for an energy balance. The objectives of analysis and generation of new approaches for general processors have been complete with the deliverable 4.3 (first part of this document). The generation of Tools for power estimation has been partially completed and have been applied only to the ARM processor because the truncation of the project.

Three energy models for instruction level power evaluation has been proposed: the *Simple Bus Cycle Energy Model*, the *Enhanced Cycle Energy Model* and the microarchitectural cycle energy model.

The first model has been full evaluated, implemented and verified with laboratory experiments. It is based on the identification of the bus cycles executed by the processor, thing that is efficiently obtained by a program called CYCLEPROCESS developed in the VIP project. From experimentation results it is generate a vector of consumption for each type of cycle, from which the consumption for each bus cycle is computed. In order to generate the previous vector and verified the global results, a power measurement instrumentation system has been designed and implemented in the UPC labs. The measurements have a deviation better than the 0.3% of the nominal value. The results of the prediction of large program consumption with the tool agree with the results obtained by the prediction tool.

The second model, the *Enhanced Cycle Energy Model*, includes the reconstruction of the cycle by cycle pipeline activity. This technique allows getting an accurate information about the temporary consumption evolution. All the tools necessary to reconstruct the pipeline activity have been implemented, but no experimental results are provided.

Finally a third model, the microarchitectural cycle model, has been investigated. This technique, that works also at cycle level, takes into consideration aspects of the processor architecture and data processed or transferred. A computing tool generates the consumption in each cycle by considering the pipeline activity for each instruction as well as a microarchitectural model for the ARM from which the set of resources activated in each cycle is computed and the corresponding consumption including data implication evaluated. This last model is able to foresee the evolution of consumption in front of power supply voltage changes and technology size evolution.

6 BIBLIOGRAPHY

ARM00a ARM, "SDT version 2.51", 2000

ATM99a ATMEL, "ATMEL Corporation ARMTDMI Datasheet", 1999

ATM00a ATMEL, "AT91EB01 Evaluation Board. Technical Reference", 2000

BOO50a A. D. Booth, "A Signed Binary Multiplication Technique", Quart. Journ. Mech. And Applied Math., Vol. IV, Pt. 2, pp. 236-240, August, 1950

BRO00a D. Brooks, V. Tiwari and M. Martonosi, "Wattch: A Framework for Architectural-Level Power Analysis and Optimizations", 27th Annual International Symposium on Computer Architecture, June, 2000

BUR92a R. Burch, F. Najm, P. Yaflg and T. Trickf, "McPOWER: A Monte Carlo Approach to Power Estimation", Proceedings of the International Conference on Computer Aided Design, pp. 90-97, November, 1992

CAL93a T. K. Callaway and E. E. Swartzalender, Jr., "Optimizing Multipliers for WSI.", Proceedings of the International Conference on Wafer Scale Integration, pp. 85-95, 1993

CAL93b T. K. Callaway and E. E. Swartzlander, Jr., "Estimating the Power Consumption of CMOS Adders", 11th Symposium on Computer Arithmetic Circuits, pp. 210-216, 1993

CHA92a A. Chandrakasan, M. Potkonjak, J. Rabaey and R. Brodersen, "An Approach for Power Minimization using transformations", IEEE Workshop on VLSI Signal Processing, pp. 41-50, June, 1992

CHA92b A. P. Chandrakasan, S. Sheng and R. W. Brodersen, "Low-Power CMOS Digital Design", IEEE Journal of Solid-State Circuits, Vol. 27, No. 4, pp. 473-483, April, 1992

CHA95c A. P. Chandrakasan, M. Potkonjak, R. Mehra, J. Rabaey and R. W. Brodersen, "Optimizing Power Using Transformations", IEEE Trans. on CAD, Vol. 14, No. 1, pp. 12-31, January, 1995

CHA99a C. Chakravarti and D. Gaitonde, "Instruction Level Power Model of Microcontrollers", Proceedings of the IEEE International Symposium on Circuits and Systems, pp. I76-I79, 1999

CHE90a K-T. Cheng and V. Agrawal, "An Entropy Measure for the Complexity of Multi-Output Boolean Functions", 27th ACM/IEEE Design Automation Conference, pp. 302-305, June, 1990

CON00a T. M. Conte, K. N. Menezes, S. W. Sathaye and M. C. Toburen, "System-Level Power Consumption Modeling and Tradeoff Analysis Techniques for Superscalar Processor Design", IEEE Transactions on Very Large Scale Integration (VLSI) Systems Vol 8, No 2, pp. 129-137, April, 2000

DAG97a J.M. Daga and D. Auvergne, "Signal Transition Modeling in Submicronic CMOS Structures", Proceedings of International Conference on PATMOS, pp. 259-268, 1997

DEN95a A.-C. Deng, X. Huang, S. Napper, J. Tuan and J. Benkoski, "Simulation Algorithms, Power Estimation and Diagnostics in PowerMill", International Workshop on Power and Timing Modeling for Performance of Integrated Circuits, pp. 399-410, 1995

FUR96a S. B. Furber, "ARM System Architecture", Addison Wesley Longman, 1996

FOR98a W. Fornaciari, P. Gubian, D. Sciuto and C. Silvano, "Power estimation of embedded systems: A hardware/software codesign approach", IEEE Transactions on VLSI Sysetms, Vol. 6, No. 2, pp. 266-275, June, 1998

GHI01a S. Ghiasi and D. Grunwald, "A Comparison of Two Architectural Power Models", Internal Report, 2001

GON96a R. Gonzalez and M. Horowitz, "Energy Dissipation In General Purpose Microprocessors", IEEE Journal of Solid-State Circuits, Vol. 31, No. 9, pp. 1277-1284, September, 1996

GU95a R. X. Gu and M. I. Elmasry, "Power Dissipation in Deep Submicron CMOS Digital Circuits", IEEE Internation Symposium on Circuits and Systems, Vol. 1, pp. 33-36, 1995

GUP00a S. Gupta and F. N. Najm, "Power Modeling for High-Level Power Estimation", IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol. 8, No. 1, pp. 18-29, February, 2000

HSI98a C. T. Hsieh and Pedram-M, "Microprocessor power estimation using profile-driven program synthesis", IEEE-Transactions-on-Computer-Aided-Design-of-Integrated-Circuits-and-Systems. vol.17, no.11, pp. 1080-1089, November, 1998

HUA95a C. X. Huang, B. Zhang, A.-C. Deng and B. Swirski, "The design and implementation of PowerMill", Proceedings of the ACM/IEEE International Symposium on Low Power Design, pp. 105-109, April, 1995

JAG90a U. Jagau, "SIMCURRENT - an efficient program for the estimation of the current flow of complex CMOS circuits.", Proceedings of the IEEE International Conference on Computer-Aided Design, pp. 396-399, November, 1990

KAN86a S. M. Kang, "Accurate Simulation of Power Dissipation in VLSI Circuits", IEEE Journal of Solid-State Circuits, Vol SC-21, No 5, pp. 889-891, October, 1986

KRO91a T. H. Krodel, "PowerPlay - fast dynamic power estimation based on logic simulation", Proceedings of the IEEE INternational Conference on Computer Design, pp. 96-100, October, 1991

LAB98a National Instruments, "LABVIEW version 5.0.1", 1998

LAN94a P. Landman and J. Rabaey, "Black-Box Capacitance Models for Architectural Power Analysis", Proceedings of the 1994 International Workshop on Low Power Design, pp. 165-170, April, 1994

LAN95a P. E. Landman and J. M. Rabaey, "Activity-Sensitive Architectural Power Analysis for the Control Path", Proceedings of the International Symposium on Low Power Design, pp. 93-98, April, 1995

LAN95b P. Landman and J. Rabaey, "Architectural power analysis: The dual bit type method", IEEE Transactions on Very Large Scale of Integrations (VLSI) Systems., Vol. 3, pp. 173-187, 1995

LAN96a P. E. Landman, "High-Level Power Estimation", IEEE Proceedings of the 1996 International Symposium on Low Power Electronics and Design, pp. 29-35, August, 1996

LEE97a M. T.-C. Lee, V. Tiwari, S. Malik and M. Fujita, "Power Analysis and Minimization Techniques for Embedded DSP Software.", IEEE Transactions on Very Large Scale Integration (VLSI) Systems., pp. 123-135, March, 1997

LID96a D. Lidsky and J. Rabaey, "Early Power Exploration: A World Wide Web Application", 33rd Design Automation Conference, pp. 27-32, June, 1996

LIU94a D. Liu and C. Svensson, "Power Consumption Estimation in CMOS VLSI Chips", IEEE Journal of Solid-State Circuits, Vol. 29, No. 6, pp. 663-670, June, 1994

MAC61a O. L. MacSorley, "High-Speed Arithmetic in Binary Computers", Proceedings of the IRE, pp. 67-91, January, 1961

MAC95a D. Marculescu, R. Marculescu and M. Pedram, "Information Theoretic Measures of Energy Consumption at Register Transfer Level", Proceedings of the International Symposium on Low Power Design, pp. 81-86, April, 1995

MAN95a H. Mehta, M. Borah, R. M. Owens and M. J. Irwin, "Accurate Estimation of Combinational Circuit Activity", Proceedings of the 32th Design Automation Conference., pp. 618-622, June, 1995

MAN98a S. Manich and J. Figueras, "A Metric for the Capacitive and Short-Circuit Transition Energy at Logic Level", Proceedings of the Power and Time Modelling of Systems, pp. 93-102, October, 1998

MAR96a S. Martin and J. Knight, "Optimizing Power in ASIC Behavioral Synthesis", IEEE Design and Test of Computers, pp. 58-70, 1996

MAT99a Math Works, "MATLAB program version 5.30", January, 1999

MEH94a R. Mehra and J. M. Rabaey, "Behavioral Level Power Estimation and Exploration", Proceedings of the International Workshop on Low Power Design, pp. 165-170, April, 1994

MUL91a K. Müller-Glaser, K. Kirsch and K. Neusinger, "Estimating Essential Design Characteristics to Support Project Planing for ASIC Design Management", IEEE International Conference on Computer-Aided Design, pp. 148-151, November, 1991

NAG73a L.W.Nagel and D.O.Pederson, "SPICE (Simulation Program with Integrated Circuit Emphasis)", University of California, Berkeley, Electronics Research Laboratory, Memorandum ERL-M 382, April, 1973

NAJ92a F. N. Najm, "Transition Density, A Stochastic Measure of Activity in Digital Circuits", Proceedings of the IEEE Design Automation Conference, pp. 644-649, June, 1992

NAJ94a F. N. Najm, "Low-Pass Filter for Computing the Transition Density in Digital Circuits", IEEE Transaction on Computer-Aided Design of Integrated Circuits and Systems, Vol 13, No. 9, pp. 1123-1131, September, 1994

NAJ94b F. N. Najm, "A Survey of Power Estimation Techniques in VLSI Circuits.", IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol. 2, No. 4, pp. 446-455, December, 1994

NAJ95a F. N. Najm, "Towards a High-Level Power Estimation Capability", Proceedings of the International Symposium on Low Power Design, pp. 87-92, April, 1995

NAN95a N. Kumar, S. Katkoori, Leo Rader and Ranga Vemuri, "Profile-Driven Behavioral Synthesis for Low-Power VLSI Systems", IEEE Design & Test of Computers, pp. 70-84, 1995

NEB96a W. Nebel and J. Mermet, "Low Power Design in Deep Submicron Electronics", Kluber Academic Publishers, NATO ASI Series, Vol. 337, August, 1996

PIP77a N. Pippenger, "Information Theory and the Complexity of Boolean Functions", Mathematical Systems Theory, vol. 10, pp. 129-167, 1977

POW90a S. Powell and P. Chau, "Estimating power dissipation of VLSI signal processing chips: The PFA Technique", Proc. IEEE Workshop on VLSI Signal Processing, vol. IV, pp. 250-259, 1990

RAB96a J. M. Rabaey and M. Pedram, "Low Power Design Methodologies", Kluber Academic Publishers, 1996

RIU00a X. Alcalde, J. Rius and J. Figueras, "Experimental Techniques to measure current, power and energy in CMOS IC's", Proceedings of DCIS conference, pp. 758-763, November, 2000

RUB75a L. P. Rubinfield, "A Proof of the Modified Booth's Algorithm for Multiplication", IEEE Transactions on Computers, pp. 1014-1015, October, 1975

SAR99a D. Sarta, D. Trifone and G. Ascia, "A Data Dependent Approach to Instruction Level Power Estimation", Proceedings of the IEEE Alessandro Volta Workshop on Low Power Design, pp. 182-190, March, 1999

SAT94a T. Sato, M. Nagamatsu and H. Tago, "Power and Performance Simulator: ESP and its Application for 100 MIPS/W Class RISC Design", IEEE Proc. of Symp. on Low Power Electronics, pp. 46-47, 1994

SAT95a T. Sato, Y. Ootaguro, M. Nagamatsu and H. Tago, "Evaluation of architecture-level power estimation for CMOS RISC processors", Proc. Symp. Low Power Electronics, pp. 44-45, October, 1995

TEK00a Tektronix, "Measurement products catalog", 2000

TIW94a V. Tiwari, S. Malik and A. Wolfe, "Power analysis of embedded software: A first step towards software power minimization.", IEEE Transactions on VLSI Systems, pp. 437-445, December, 1994

TIW98a V. Tiwari and M. T.-C. Lee, "Power Analysis of a 32-bit Embedded Microcontroller", IEEE VLSI Design Journal, Vol. 7, No. 3, 1998

TJA89a R. Tjärnström, "Power dissipation estimate by switch level simulation", Proceedings of the IEEE International Symposium on Circuits and Systems, Vol. 2, pp. 881-884, May, 1989

TUR95b S. Turgis, N. Azemard and D. Auvergne, "Short-Circuit Power Dissipation Calculation on CMOS Inverters Using the Equivalent Short-Circuit Capacitance Concept", International Workshop on Power and Timing Modeling for Performance of Integrated Circuits, pp. 213-224, 1995

WAT96a Sente Corp., "WattWatcher Product Sheet", Chlmsford, MA., 1996

XAK94a M. Xakellis and F. Najm, "Statistical Estimation of the Switching Activity in Digital Circuits.", Proceedings of the 31th ACM/ IEEE Design Automation Conference, pp. 728-733, June, 1994

YAC89a G. Y. Yacoub and W. H. Ku, "An Enhanced Technique for Simulating Short-Circuit Power Dissipation", IEEE Journal of Solid-State Circuits, Vol. 24, No 3, pp. 844-847, June, 1989